

CSE 4303

Data Structures

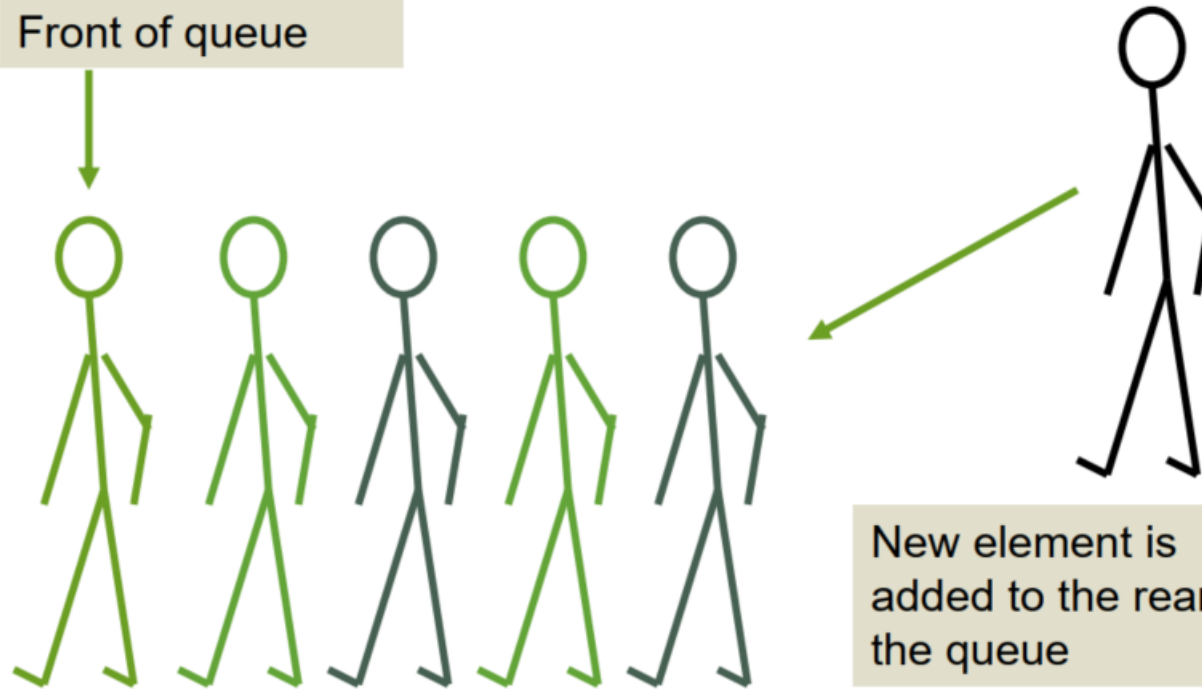
Topic: Queue

Sabbir Ahmed
Assistant Prof | CSE | IUT
sabbirahmed@iut-dhaka.edu



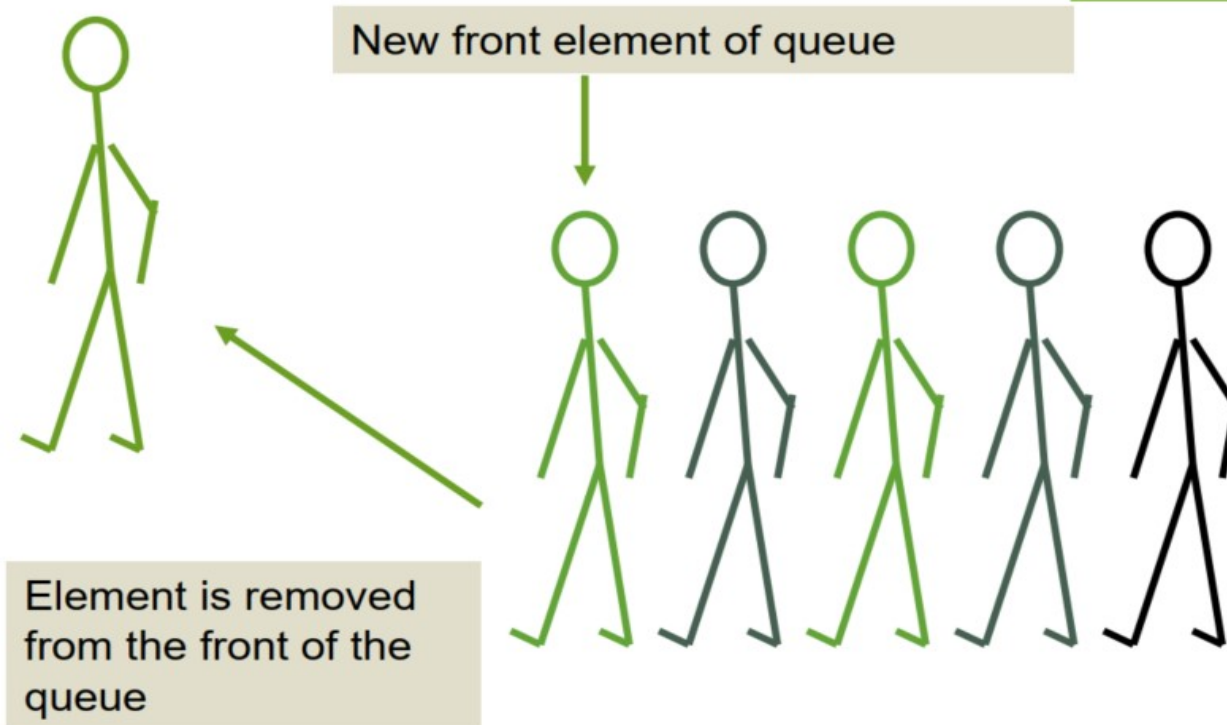
Adding an element

Front of queue



New element is added to the rear of the queue

Removing an element



Queue

- **First-in-first-out (FIFO)** data structure.
 - The item inserted first is the first item to leave.

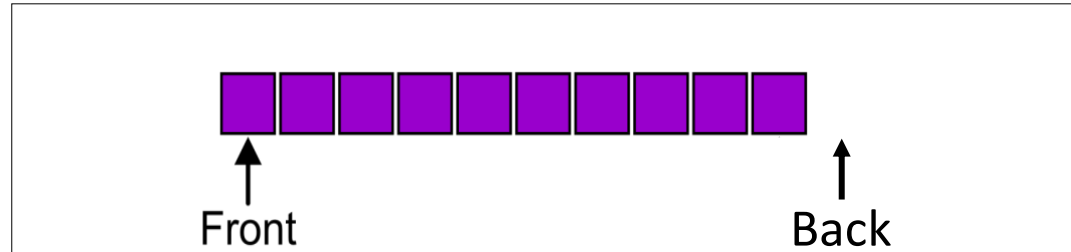
Queue

- **First-in-first-out (FIFO)** data structure.
 - The item inserted first is the first item to leave.
- Queue is a list of items-
 - Items are **deleted** from one end (**front**)
 - Items are **inserted** at the other end (**rear** or **back**)

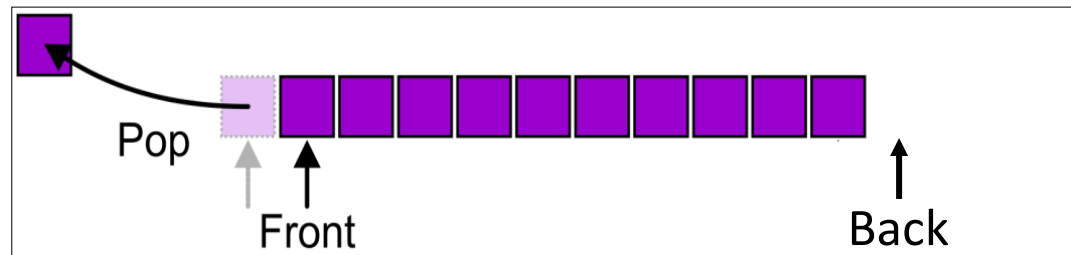
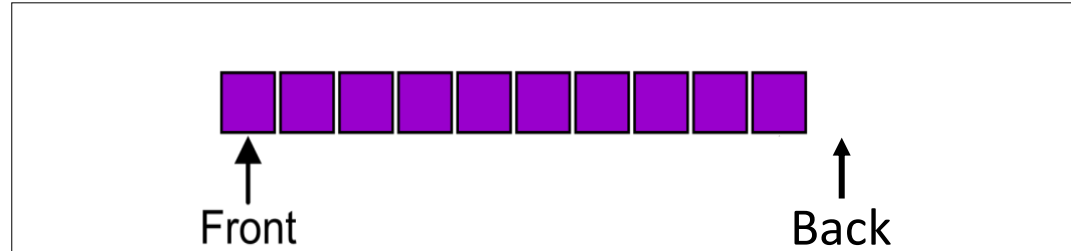
Queue

- **First-in-first-out (FIFO)** data structure.
 - The item inserted first is the first item to leave.
- Queue is a list of items-
 - Items are **deleted** from one end (**front**)
 - Items are **inserted** at the other end (**rear** or **back**)
- **Front** of Queue: an item that is in the queue for the longest amount of time.

Queue

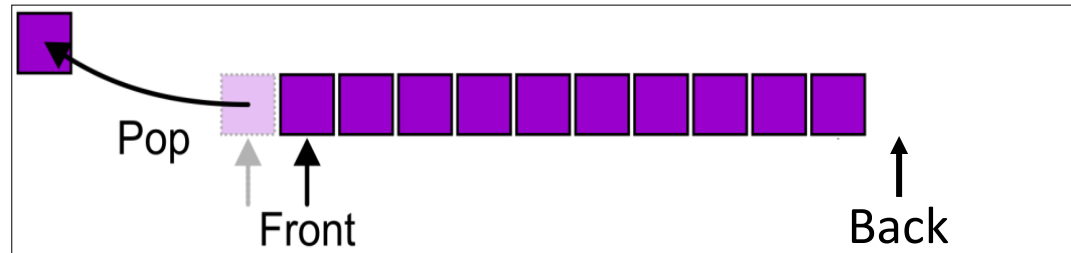
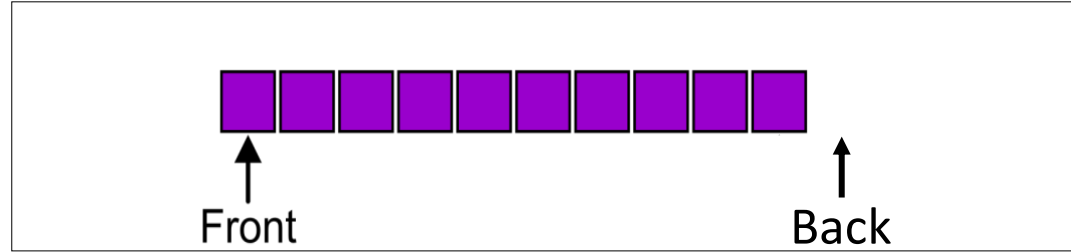


Queue



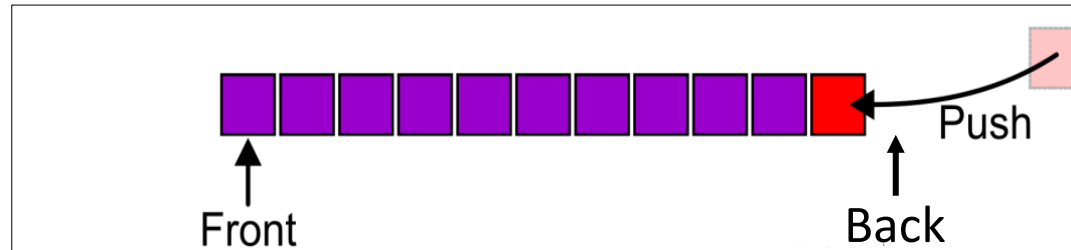
*Deleting from
Queue ~
'Dequeue'*

Queue

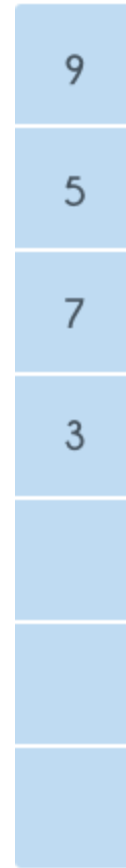


*Deleting from
Queue ~
'Dequeue'*

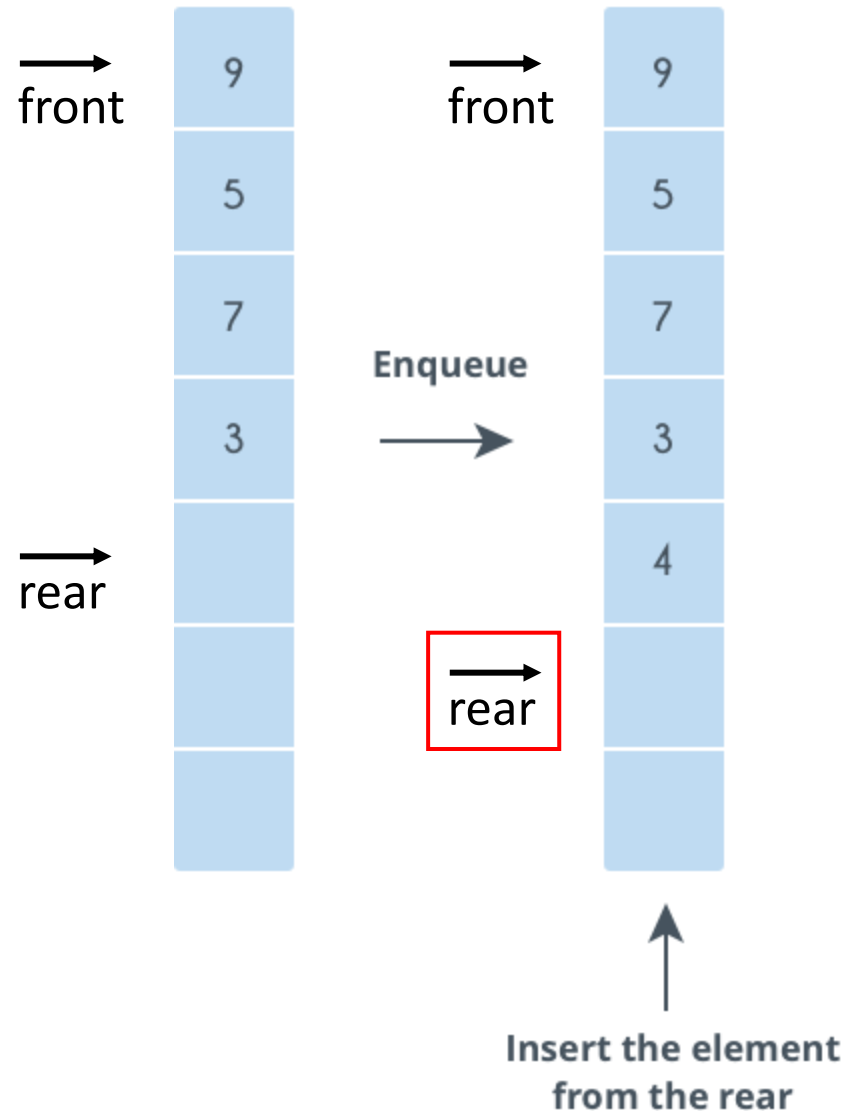
*Inserting in
Queue ~
'Enqueue'*

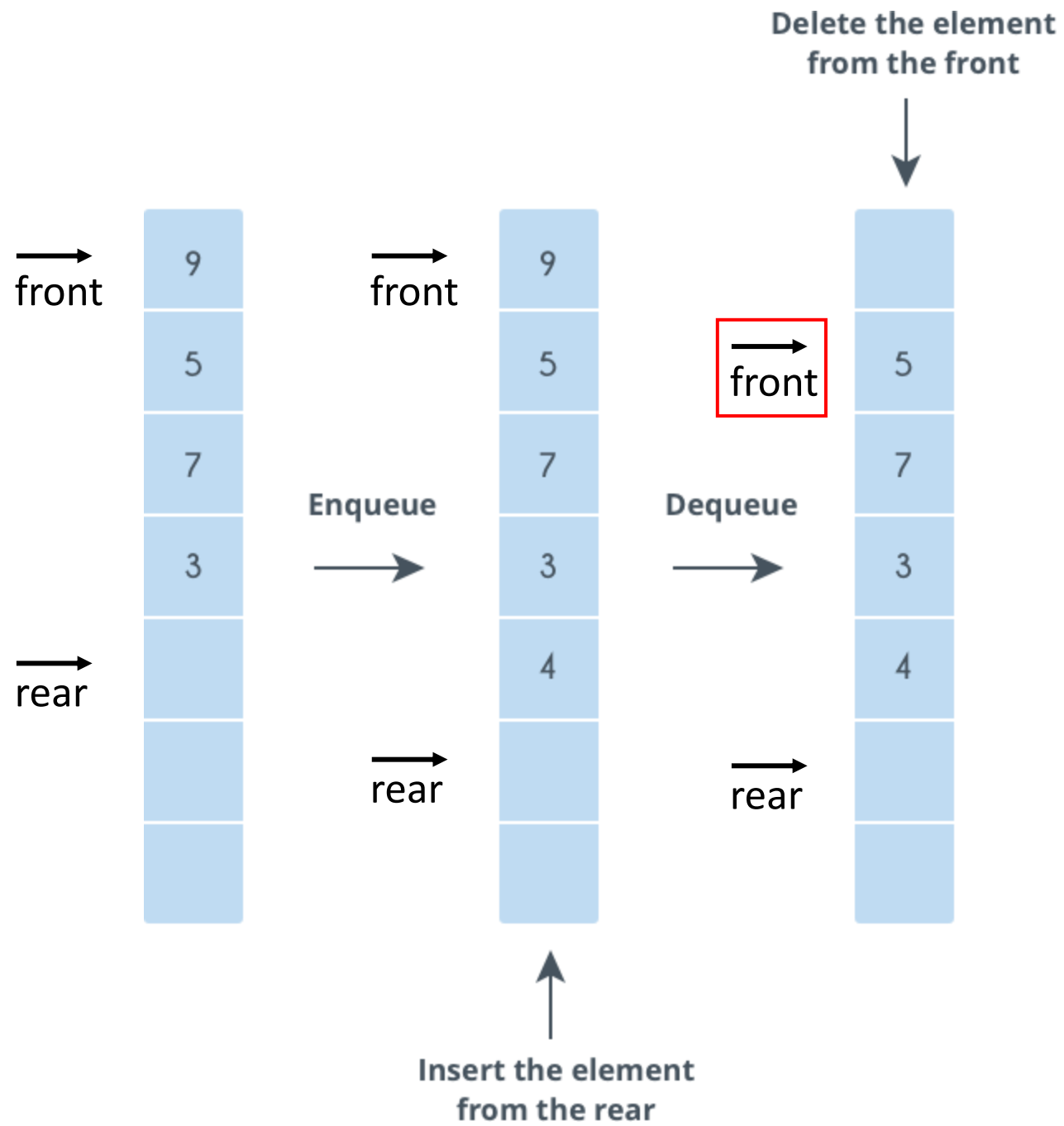


→
front



→
rear





Applications

- **Client-server models**

- Multiple clients may request services from one or more servers.
- Some clients may have to wait while the servers are busy.
- Clients are placed in a queue and served in the order of arrival.

Applications

- **Client-server models**

- Multiple clients may request services from one or more servers.
- Some clients may have to wait while the servers are busy.
- Clients are placed in a queue and served in the order of arrival.

- **Printer shared** in a network across different machines

- Maintain a queue of the documents and print one after another.

Applications

- **Client-server models**

- Multiple clients may request services from one or more servers.
- Some clients may have to wait while the servers are busy.
- Clients are placed in a queue and served in the order of arrival.

- **Printer shared** in a network across different machines

- Maintain a queue of the documents and print one after another.

- The **processor of the computer** is also a shared resource.

- Lots of programs need time from the processor, but it can attend to one process at a time (assuming single-core).
- Maintain queue of processes!

Array Implementation

Queue[]	Stores items
---------	--------------

Array Implementation

Queue[]	Stores items
maxQ	Maximum capacity of Queue[]

Array Implementation

Queue[]	Stores items
maxQ	Maximum capacity of Queue[]
Front/head	Points at the index of <u><i>next deletion</i></u>

Array Implementation

Queue[]	Stores items
maxQ	Maximum capacity of Queue[]
Front/head	Points at the index of <u>next deletion</u>
Rear/tail	Points at the index of <u>next insertion</u>

Array Implementation

All Queue operations must run in $O(1)$

Queue[]	Stores items
maxQ	Maximum capacity of Queue[]
Front/head	Points at the index of <u>next deletion</u>
Rear/tail	Points at the index of <u>next insertion</u>

}



```
void enqueue(int queue[],    int item, int &rear,  
             int maxQ)  
{  
  
    queue[rear] = item;  
    rear++;  
  
}
```

```
void enqueue(int queue[], int item, int &rear,  
            int maxQ)
```

```
{
```

*What if the Queue
is full?*

```
    queue[rear] = item;  
    rear++;
```

```
}
```

```
void enqueue(int queue[],    int item, int &rear,
              int maxQ)
{
    if (rear == maxQ)
        print( 'Overflow' )
    else{
        queue[rear] = item;
        rear++;
    }
}
```

```
void enqueue(int queue[],    int item, int &rear,  
            int maxQ)  
{  
    if (rear == maxQ)  
        print( 'Overflow' )  
    else{  
        queue[rear] = item;  
        rear++;  
    }  
}
```

*What should be
the initial values of
front & rear?*

```
void enqueue(int queue[], int item, int &rear,  
            int maxQ)  
{  
    if (rear == maxQ)  
        print( 'Overflow' )  
    else{  
        queue[rear] = item;  
        rear++;  
    }  
}
```

*What should be
the initial values of
front & rear?*

*front=0
rear=0*

```
void enqueue(int queue[], int item, int &rear,
             int maxQ)
{
    if (rear == maxQ)
        print( 'Overflow' )
    else{
        queue[rear] = item;
        rear++;
    }
}
```

*What if the time
complexity of
EnQueue?*

```
void enqueue(int queue[], int item, int &rear,
            int maxQ)
{
    if (rear == maxQ)
        print( 'Overflow' )
    else{
        queue[rear] = item;
        rear++;
    }
}
```

*What if the time
complexity of
EnQueue?
 $O(1)$*

```
Void dequeue(int queue[], int &front, int rear)
{

}
}
```

```
Void dequeue(int queue[], int &front, int rear)
{
```

```
    front++
```

```
}
```

```
Void dequeue(int queue[], int &front, int rear)
{
```

*What if there is
nothing to delete?*

```
    front++
```

```
}
```

```
Void dequeue(int queue[], int &front, int rear)
{
    If Queue is empty...
        print( 'underflow' )

    else{
        front++
    }
}
```

```
Void dequeue(int queue[], int &front, int rear)
{
    If Queue is empty...
        print( 'underflow' )

    else{
        front++
    }
}
```

*When is the Queue
empty?*

```
Void dequeue(int queue[], int &front, int rear)
{
    If Queue is empty...
        print( 'underflow' )

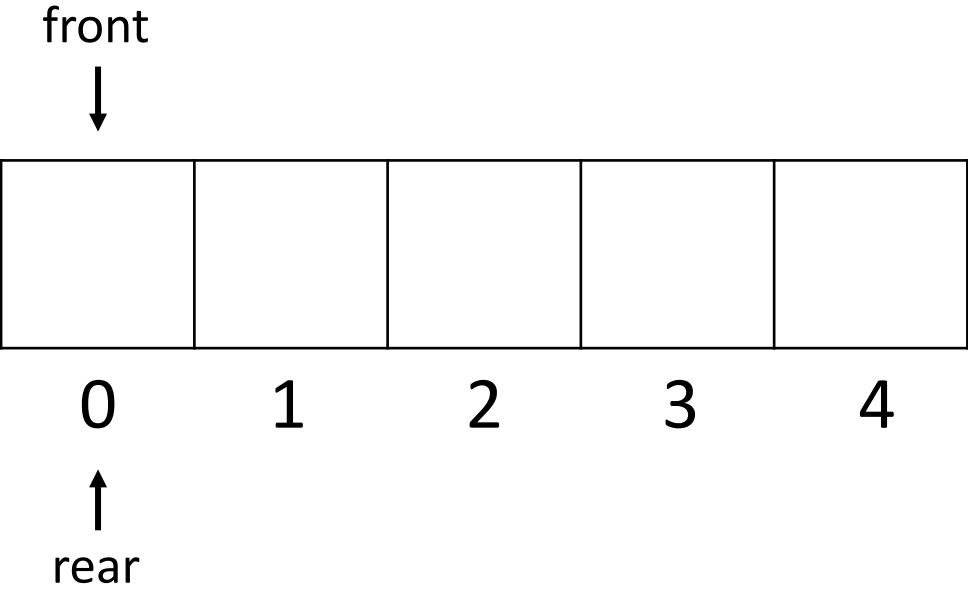
    else{
        front++
    }
}
```

*When is the Queue
empty?
front==rear*

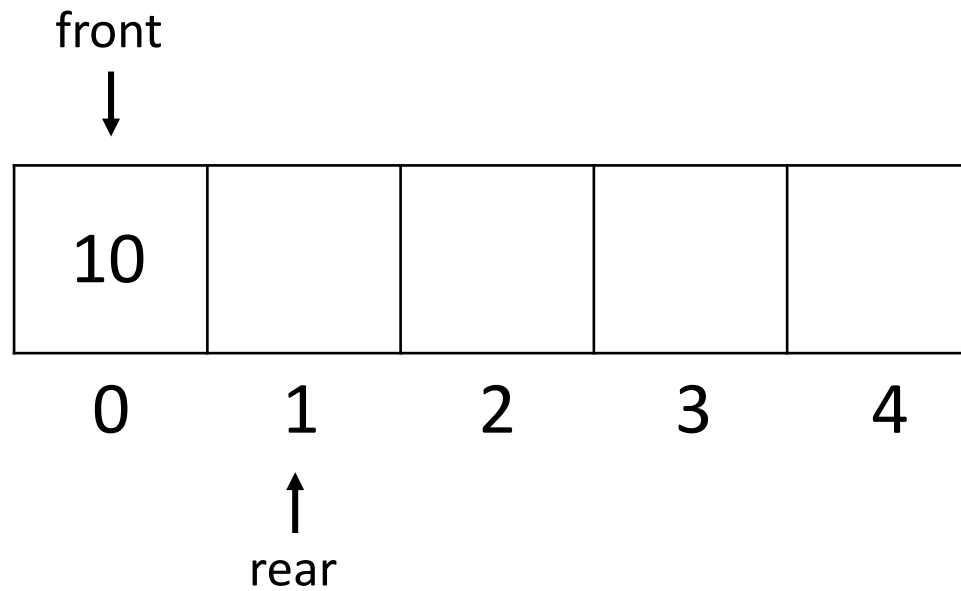
```
Void dequeue(int queue[], int &front, int rear)
{
    if(front == rear)
        print('underflow')

    else{
        front++
    }
}
```

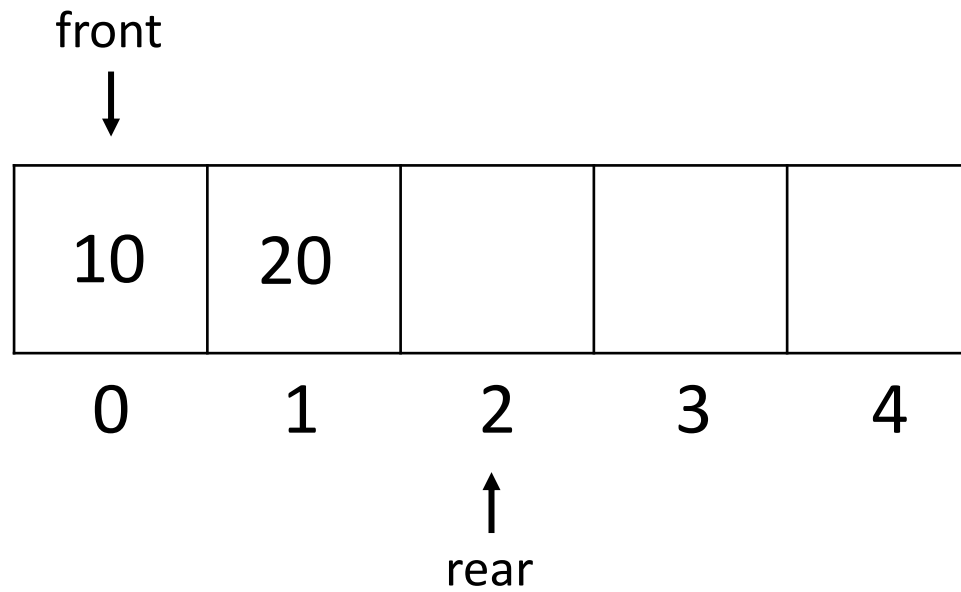
Let's take an empty Queue
 $\text{maxQ}=5$



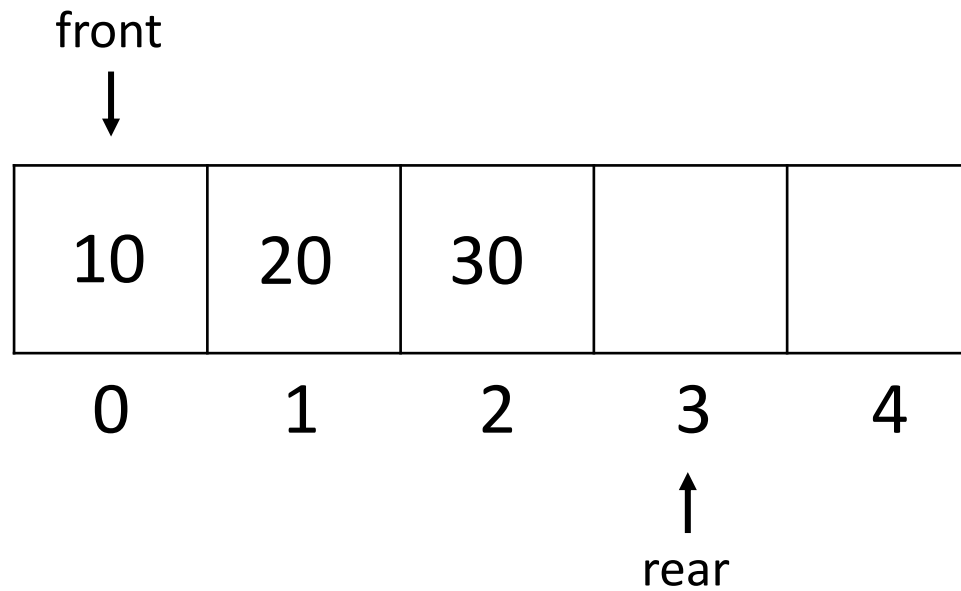
Enqueue(10)



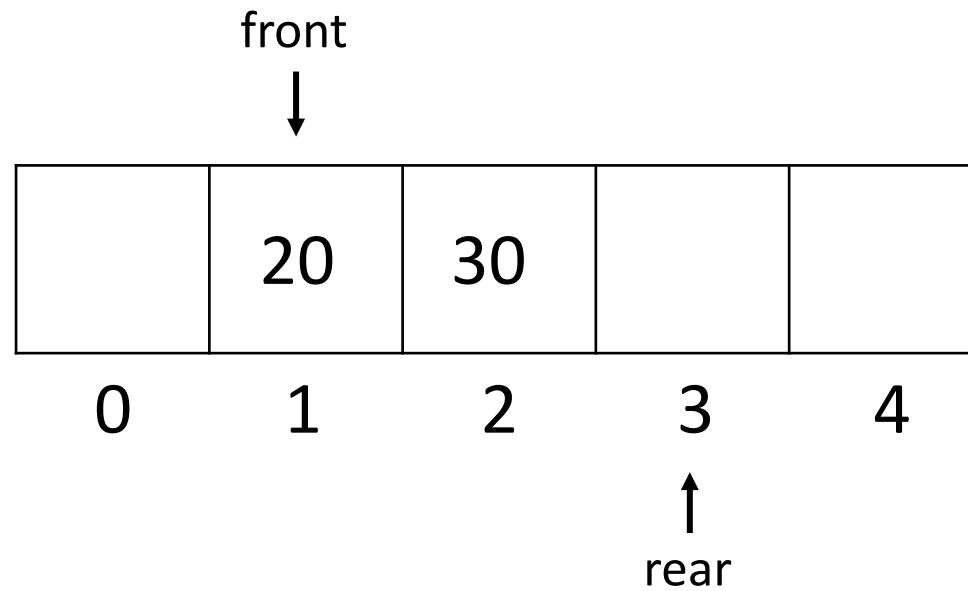
Enqueue(20)



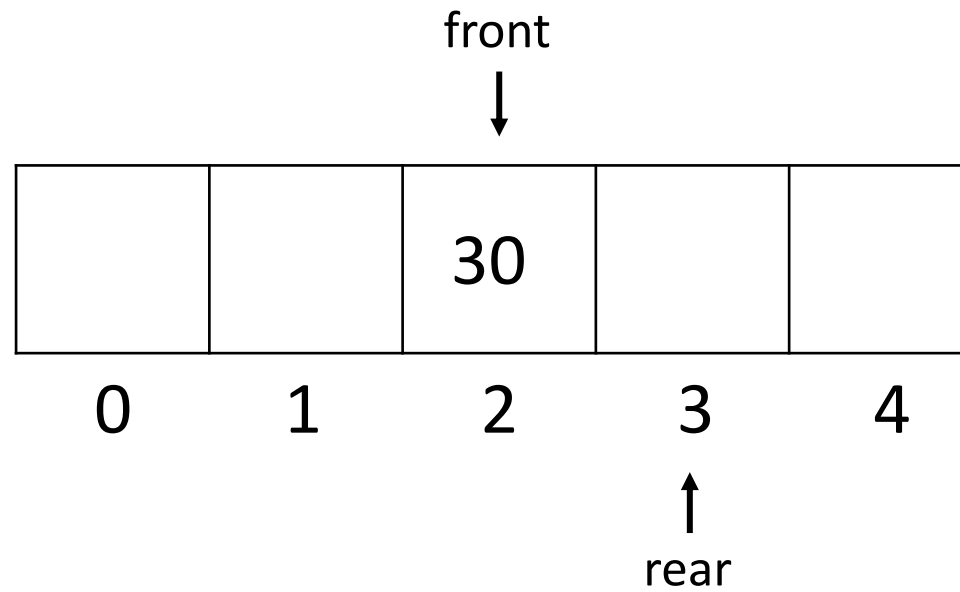
Enqueue(30)



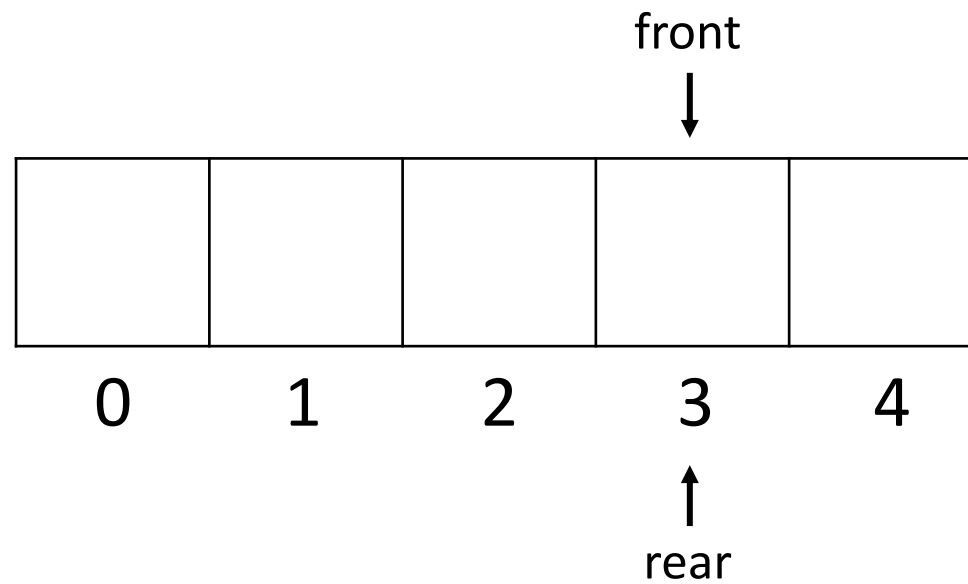
Dequeue()



Dequeue()



Current status



```
int front( int queue[], int front){  
    return queue[front];  
}
```

```
int front( int queue[], int front){  
    return queue[front];  
}
```

```
int size( int front, int rear){  
    return ???  
}
```

```
int front( int queue[], int front){  
    return queue[front];  
}
```

```
int size( int front, int rear){  
    return (rear-front);  
}
```

```
int front( int queue[], int front){  
    return queue[front];  
}
```

```
int size( int front, int rear){  
    return (rear-front);  
}
```

```
bool isEmpty( int front, int rear){  
    return ???  
}
```

```
int front( int queue[], int front){  
    return queue[front];  
}
```

```
int size( int front, int rear){  
    return (rear-front);  
}
```

```
bool isEmpty( int front, int rear){  
    return (front == rear);  
}
```

Linked-list implementation

- In a singly linked list, $\theta(1)$ **insertion** is possible at both the front and the back.

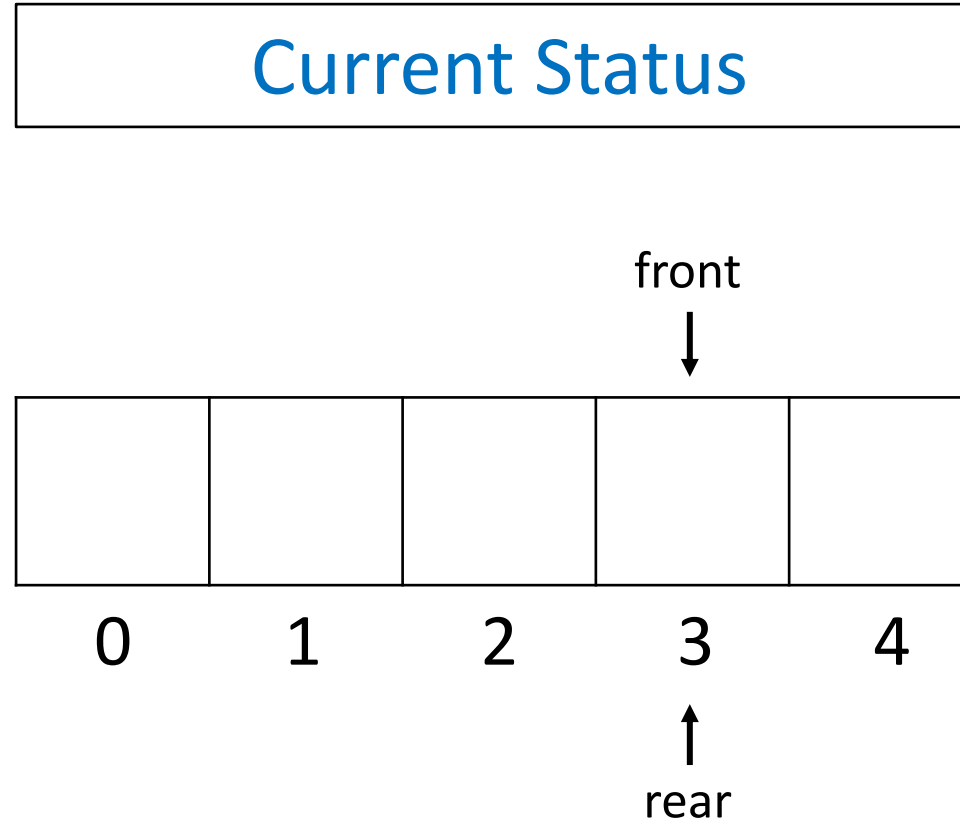
Linked-list implementation

- In a singly linked list, $\theta(1)$ **insertion** is possible at both the front and the back.
- But a $\theta(1)$ **erase** is only possible at the front.

Linked-list implementation

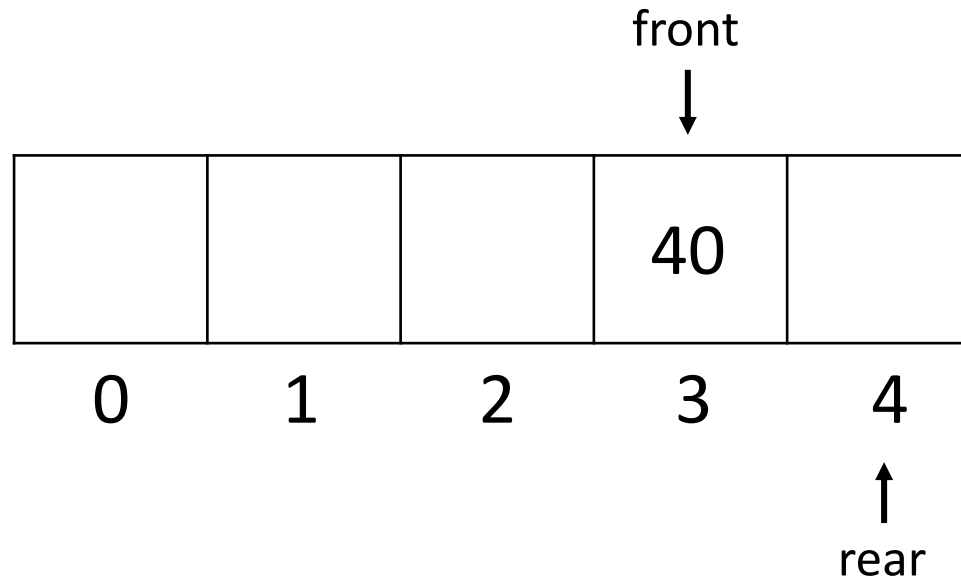
- In a singly linked list, $\theta(1)$ **insertion** is possible at both the front and the back.
- But a $\theta(1)$ **erase** is only possible at the front.
- To satisfy the behavior of queue, insertions will be at the back while erases are from the front.

Problems of one-ended array

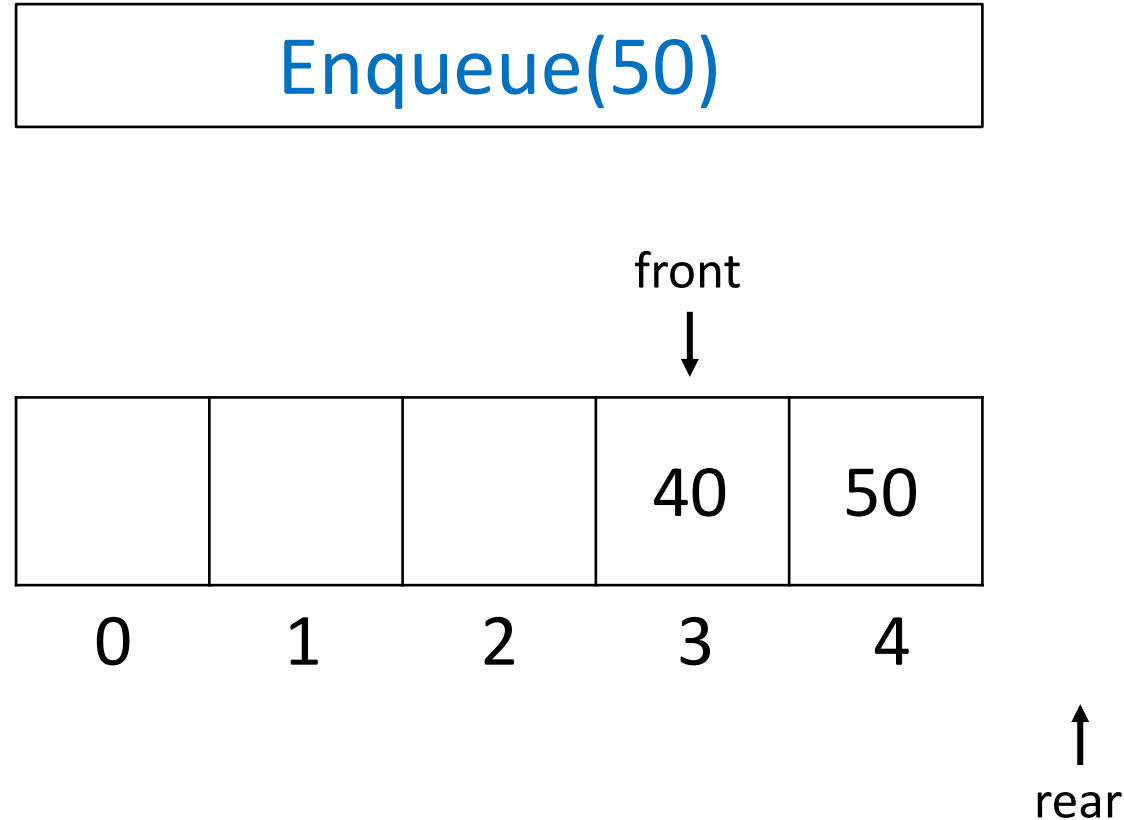


Problems of one-ended array

Enqueue(40)



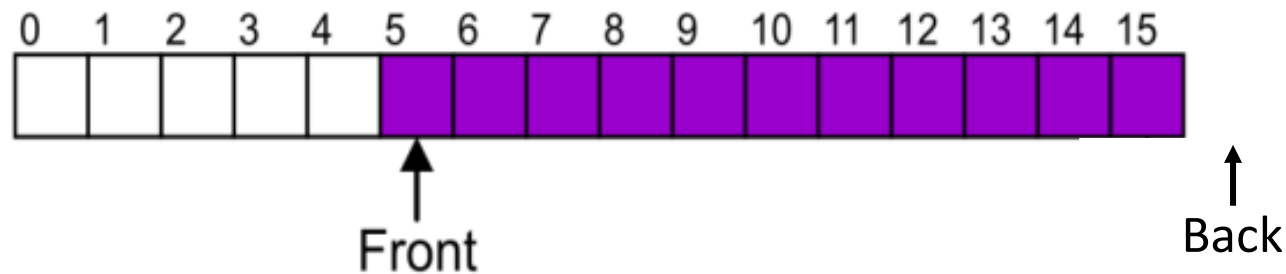
Problems of one-ended array



*Though the Queue isn't full,
can't place any more items!!*

Problems of one-ended array

- Suppose the capacity of a Queue is 16.
 - Performed 16 enqueue and 5 dequeue operations.
 - Queue size is now 11.



*Though the Queue isn't full,
can't place any more items!!*

Circular array

- Instead of viewing the array in the range of 0....15, consider the indices being cyclic:

.., 15, 0, 1, .., 15, 0, 1, .., 15, 0, 1, ..

Circular array

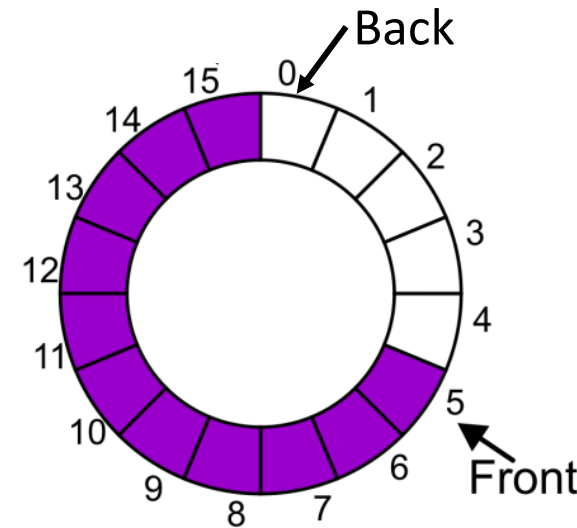
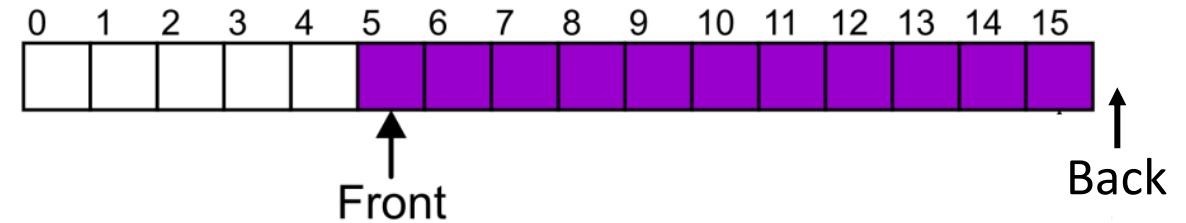
- Instead of viewing the array in the range of 0....15, consider the indices being cyclic:

.., 15, 0, 1, .., 15, 0, 1, .., 15, 0, 1, ..

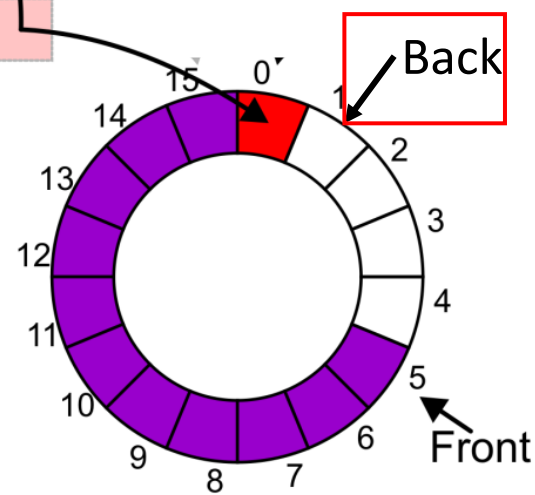
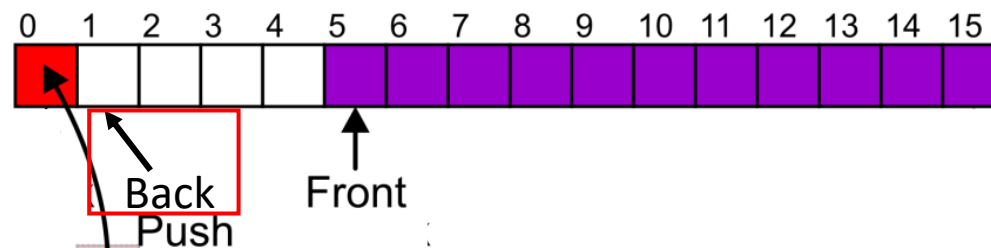
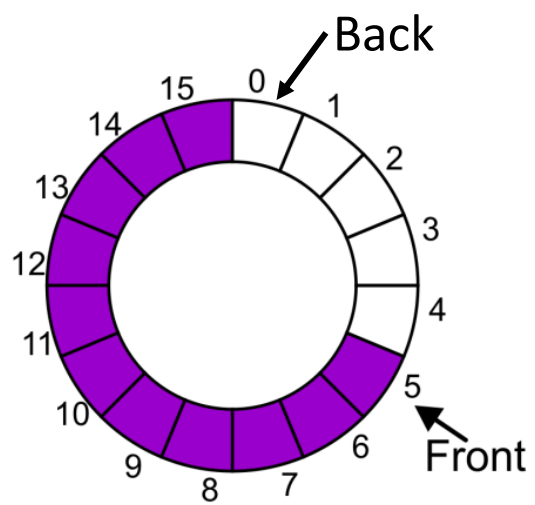
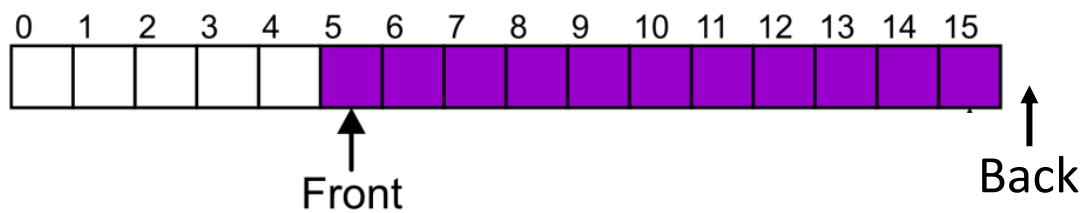
- This idea is referred to as a ***circular array***.
- Push will be performed in the next available location.

Circular array

- Instead of viewing the array in the range of 0....15, consider the indices being cyclic:
.., 15, 0, 1, .., 15, 0, 1, .., 15, 0, 1, ..
- This idea is referred to as a ***circular array***.
- Push will be performed in the next available location.

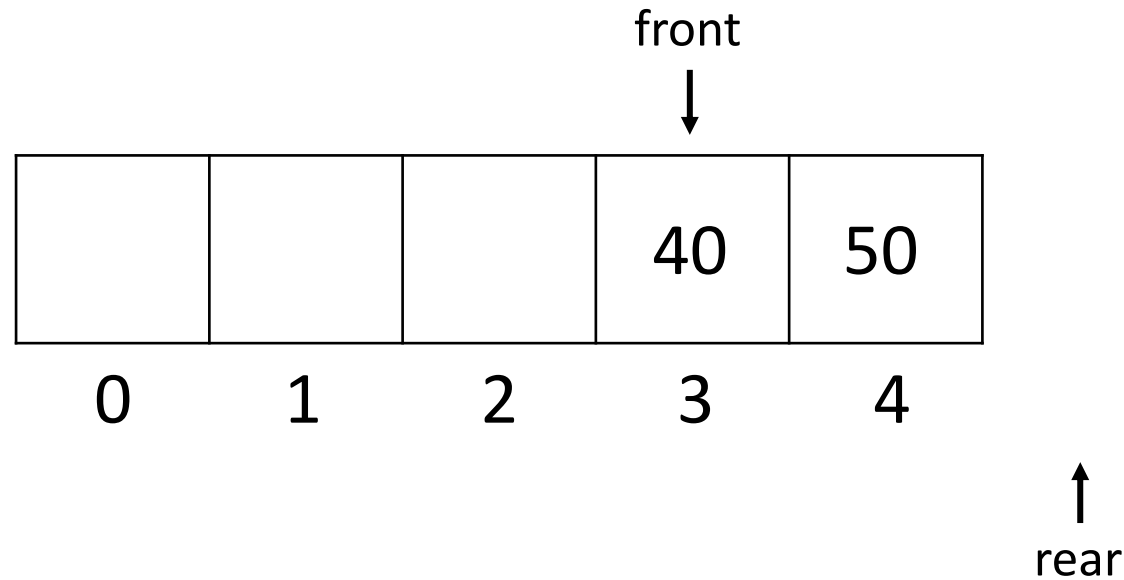


*Conceptual
view, not
physically
connected!*



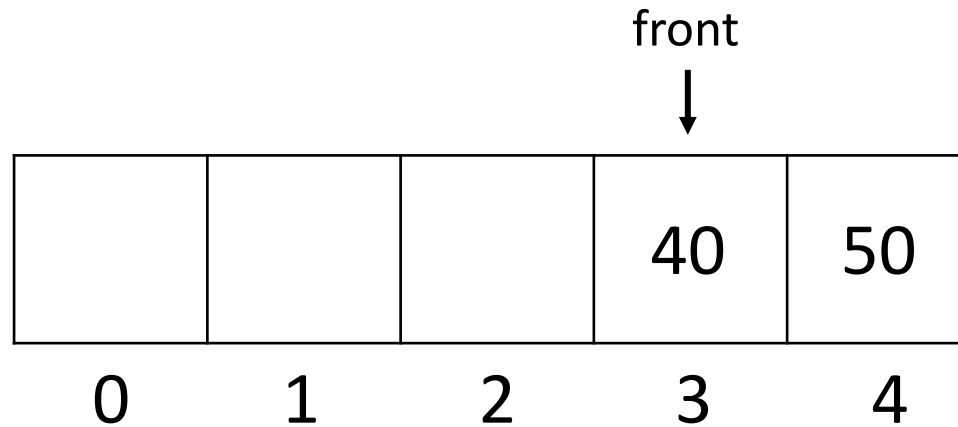
Circular array

Current status



Circular array

Current status

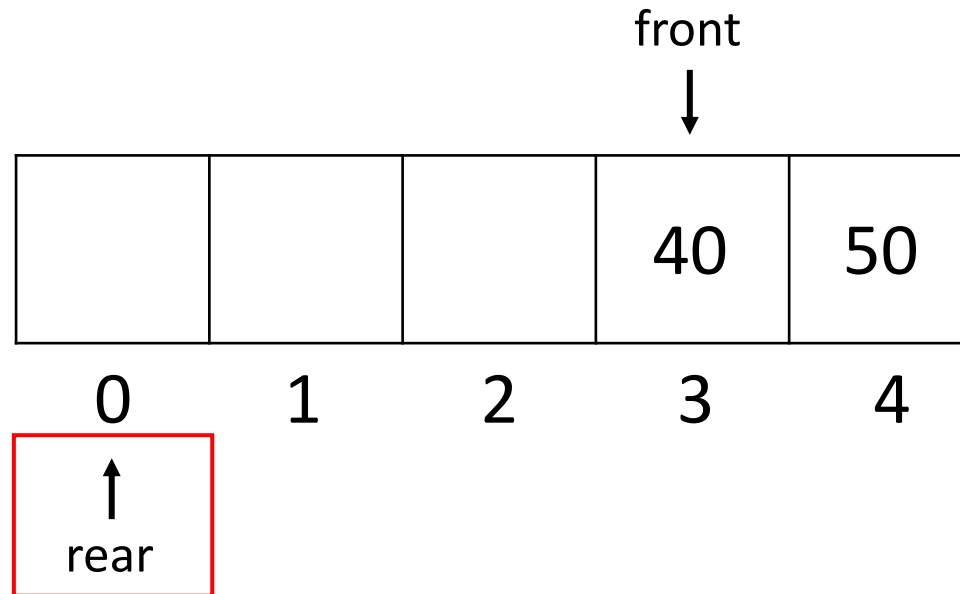


*This can't happen in
a circular array.
After Enqueue(50)
rear will go back to 0*

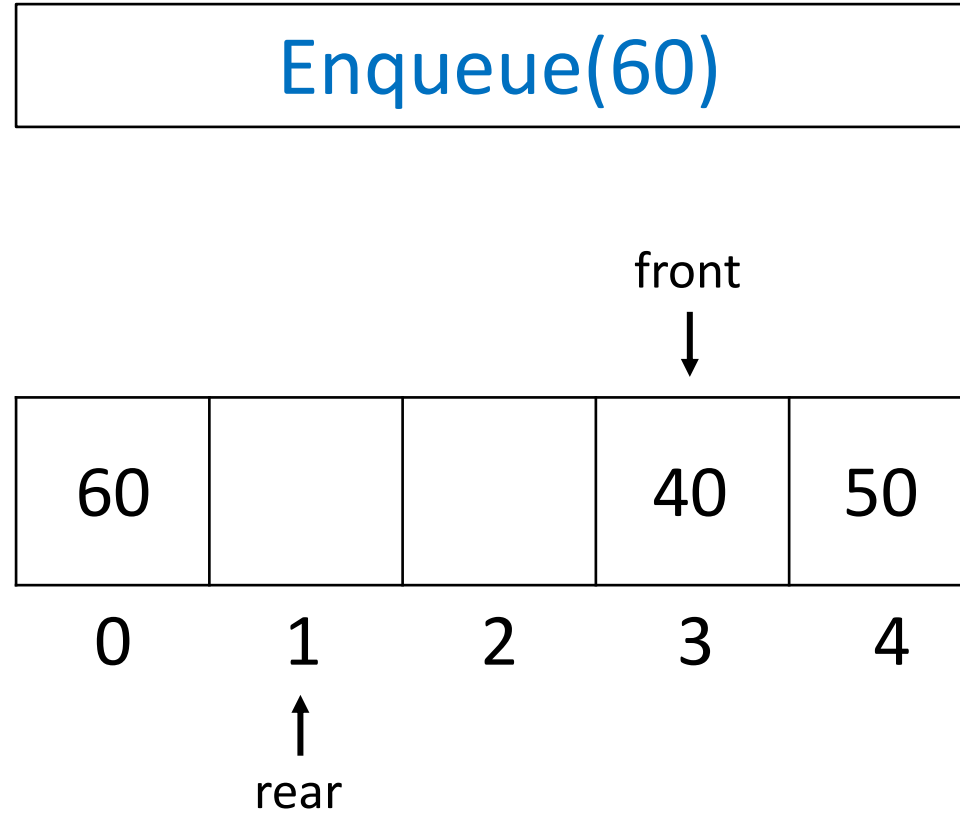
↑
rear

Circular array

Current status

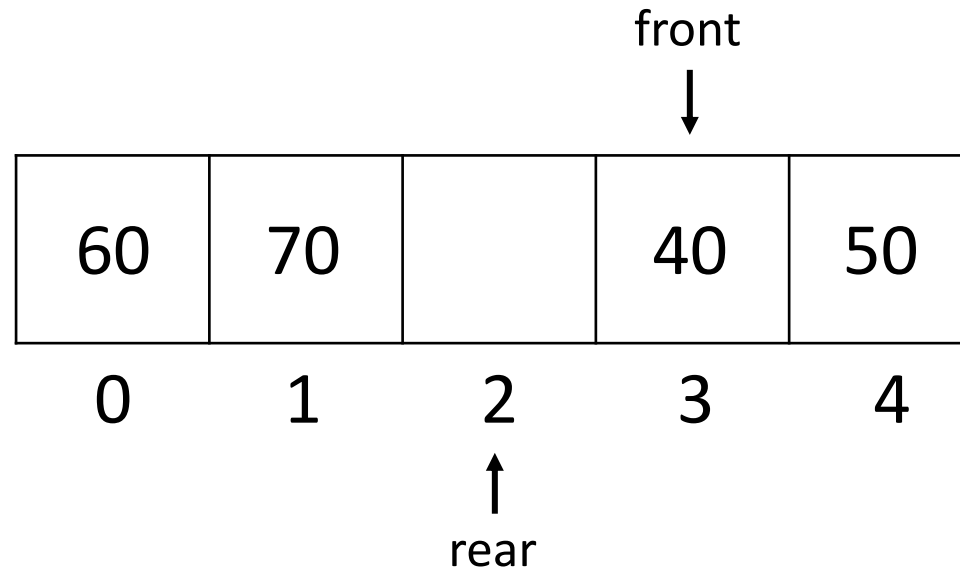


Circular array

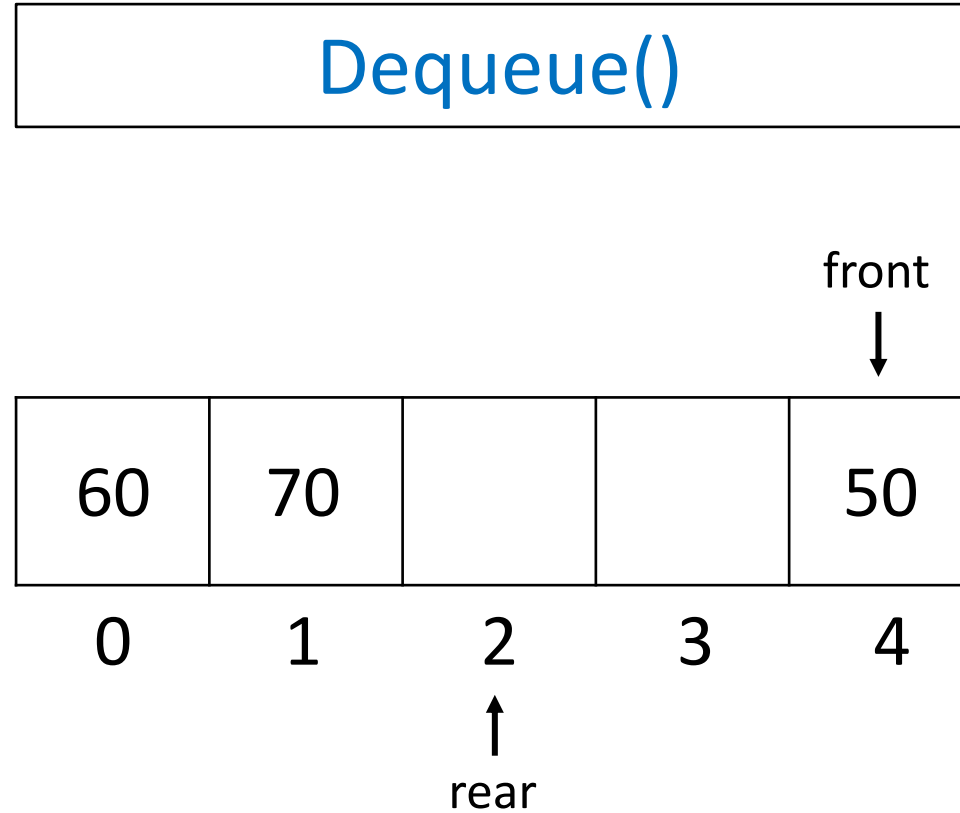


Circular array

Enqueue(70)

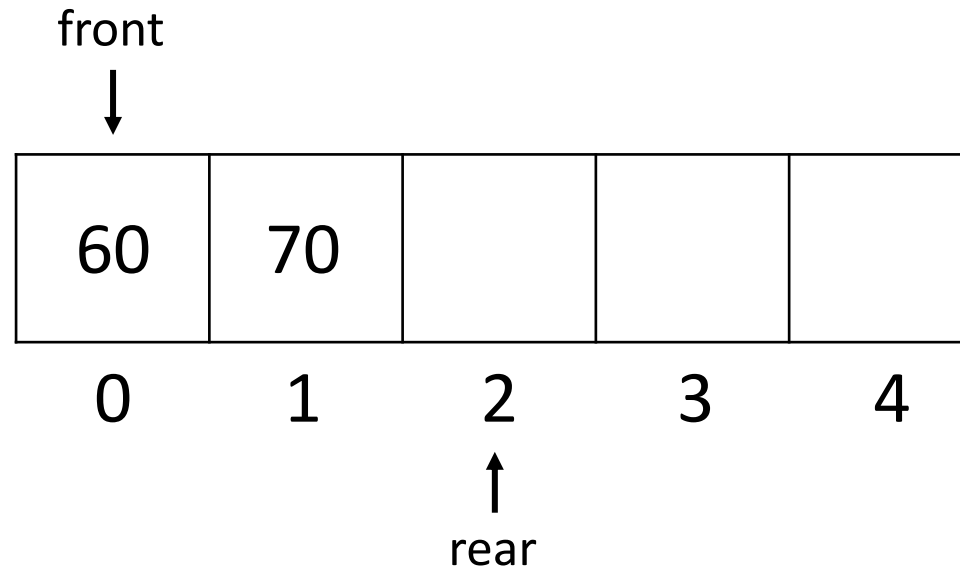


Circular array



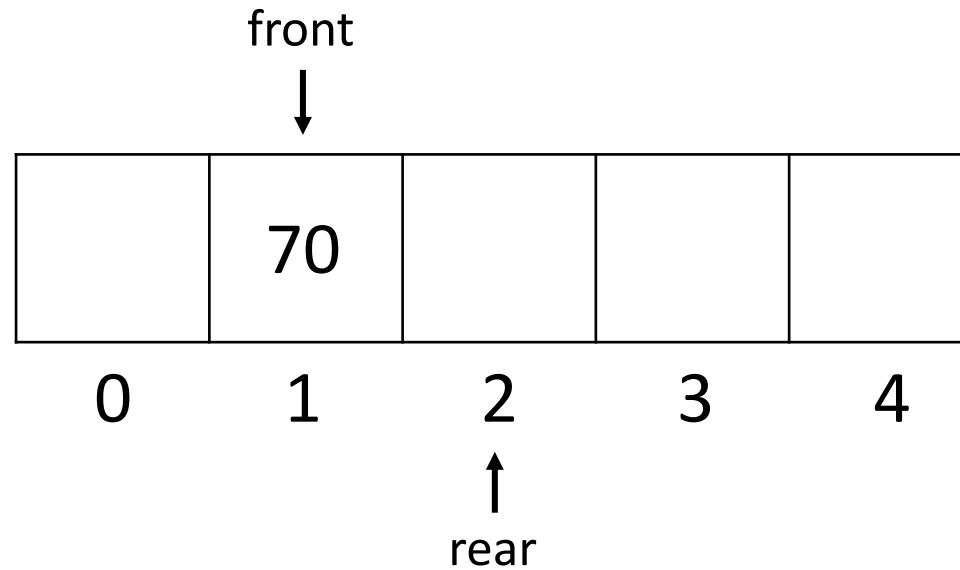
Circular array

Dequeue()

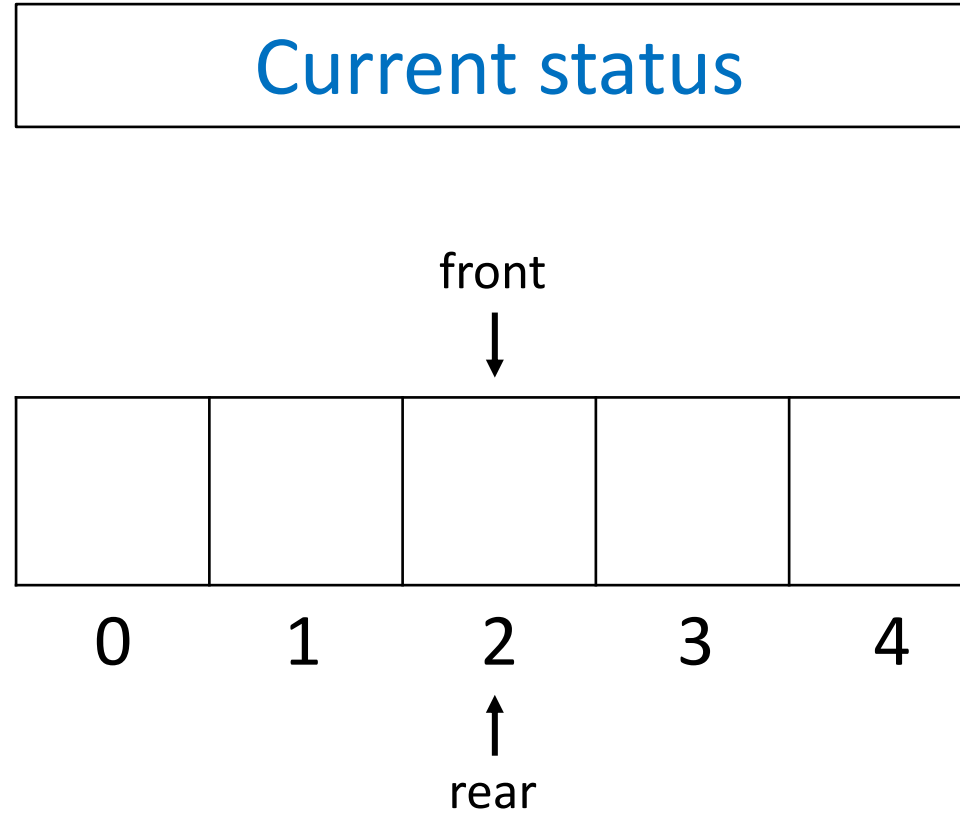


Circular array

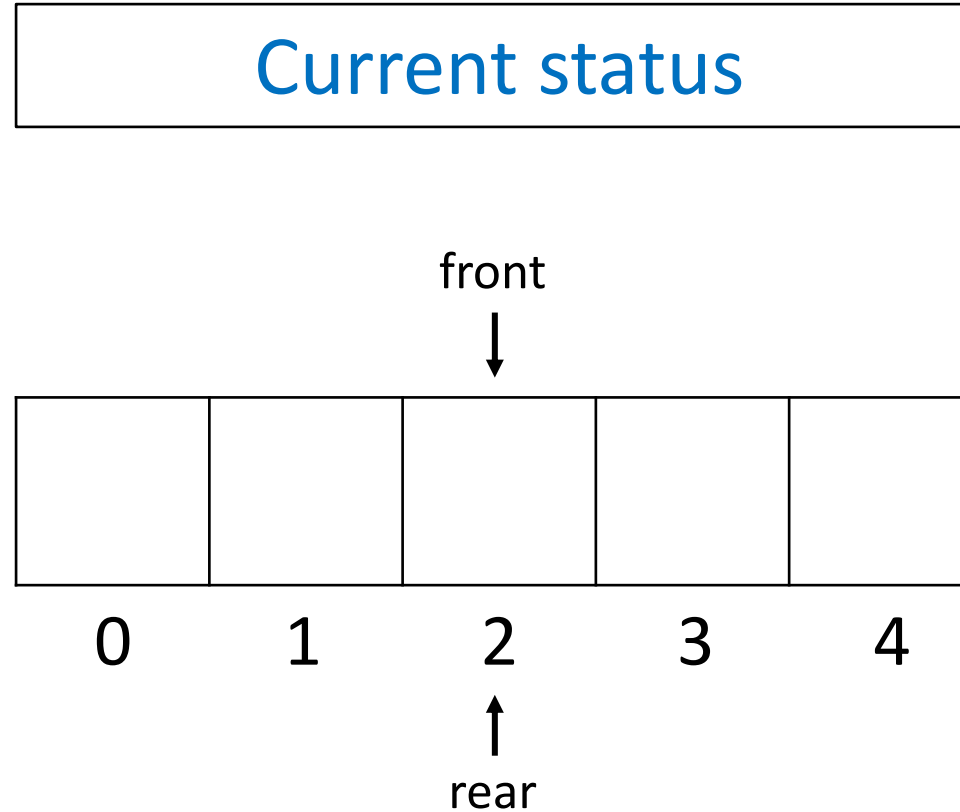
Dequeue()



Circular array

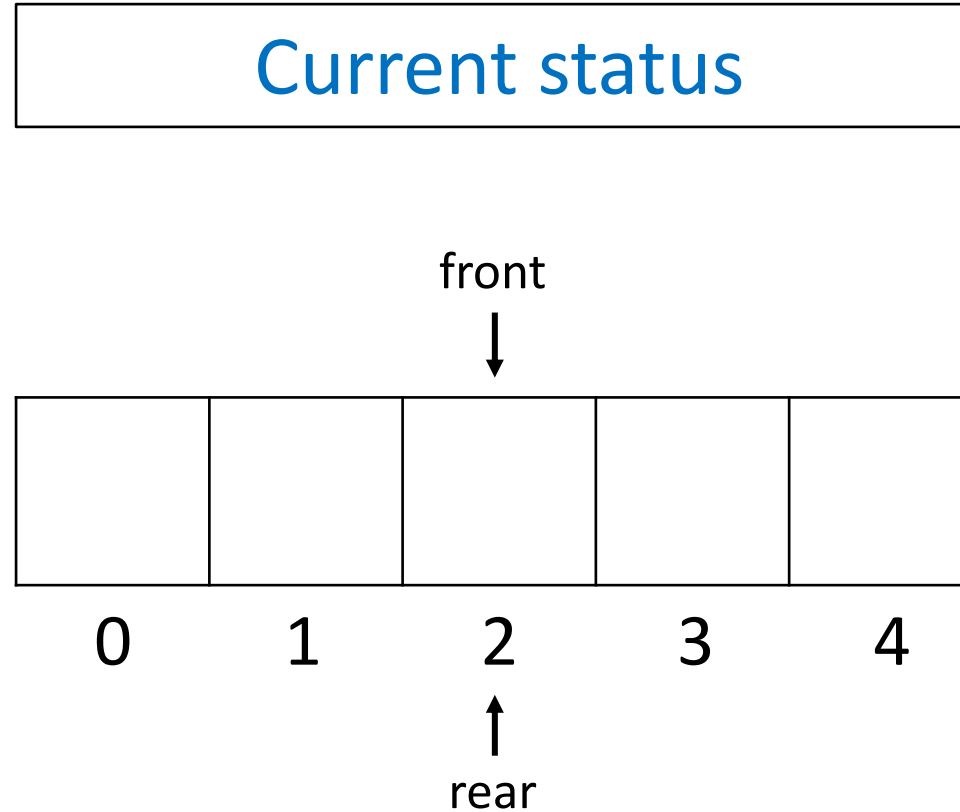


Circular array



*Can we assume
the queue is
empty when
 $front == rear$?*

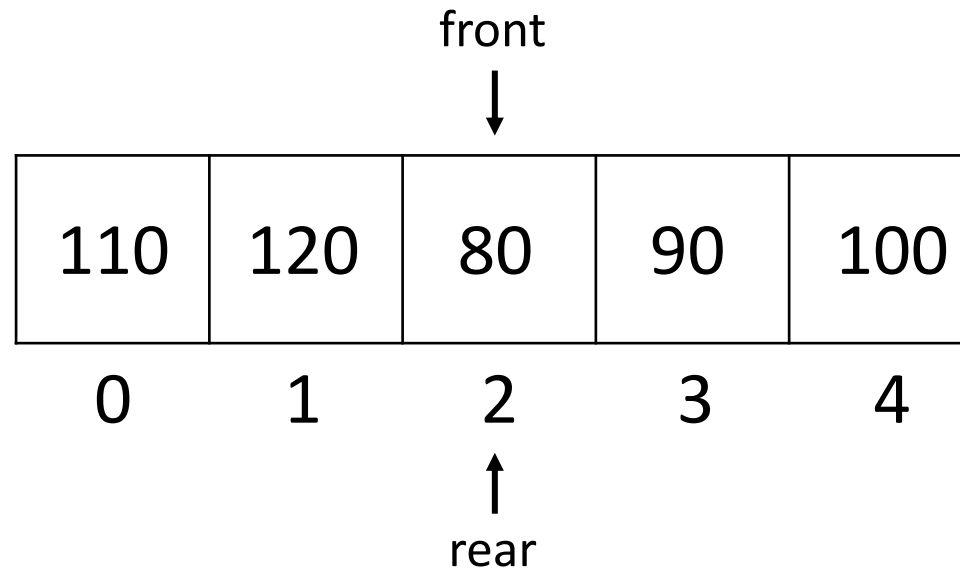
Circular array



*Can we assume
the queue is
empty when
 $front == rear$?
Not always!!*

Circular array

Enqueue(80)
Enqueue(90)
Enqueue(100)
Enqueue(110)
Enqueue(120)



*In a Circular array,
front==rear
can also appear
when the Queue is
full !!*

```
void circular_enqueue (int queue[], int item,  
                      int rear, int maxQ, int &count)  
{
```

```
void circular_enqueue (int queue[], int item,  
                      int rear, int maxQ, int &count)  
{  
    if( count == maxQ ) // Queue full  
        print( 'Overflow' );  
  
    else{  
        queue[rear] = item;  
          
        count++;  
    }  
}
```

*For simplicity, we've
used the 'count'
variable that keeps
the current size of
the Queue*

```
void circular_enqueue (int queue[], int item,  
                      int rear, int maxQ, int &count)  
{  
    if( count == maxQ ) // Queue full  
        print( 'Overflow' );  
  
    else{  
        queue[rear] = item;  
        rear = (rear+1) % maxQ;  
        count++;  
    }  
}
```

```
void circular_dequeue(int queue[], int &front,  
                      int rear, int &count)  
{
```

```
void circular_dequeue(int queue[], int &front,  
                      int rear, int &count)  
{  
    if( count == 0)  
        print( 'Underflow' )  
  
    else{  
  
        count--;  
    }  
}
```

```
void circular_dequeue(int queue[], int &front,  
                      int rear, int &count)  
{  
    if( count == 0)  
        print( 'Underflow' )  
  
    else{  
        front = (front+1) % maxQ  
        count--;  
    }  
}
```

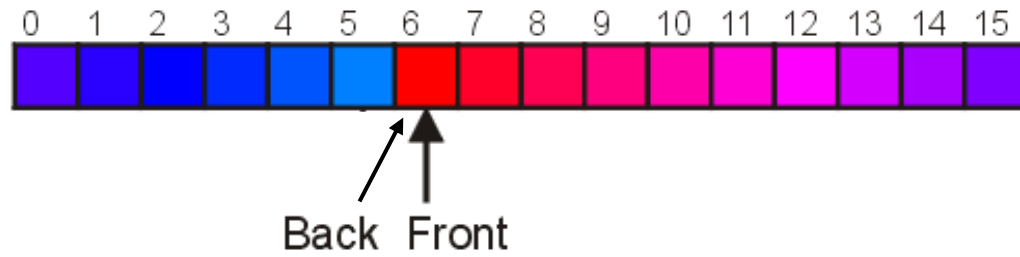
Size(): returns count

// the number of items present in the
Queue at any moment

isEmpty: returns True, if count=0

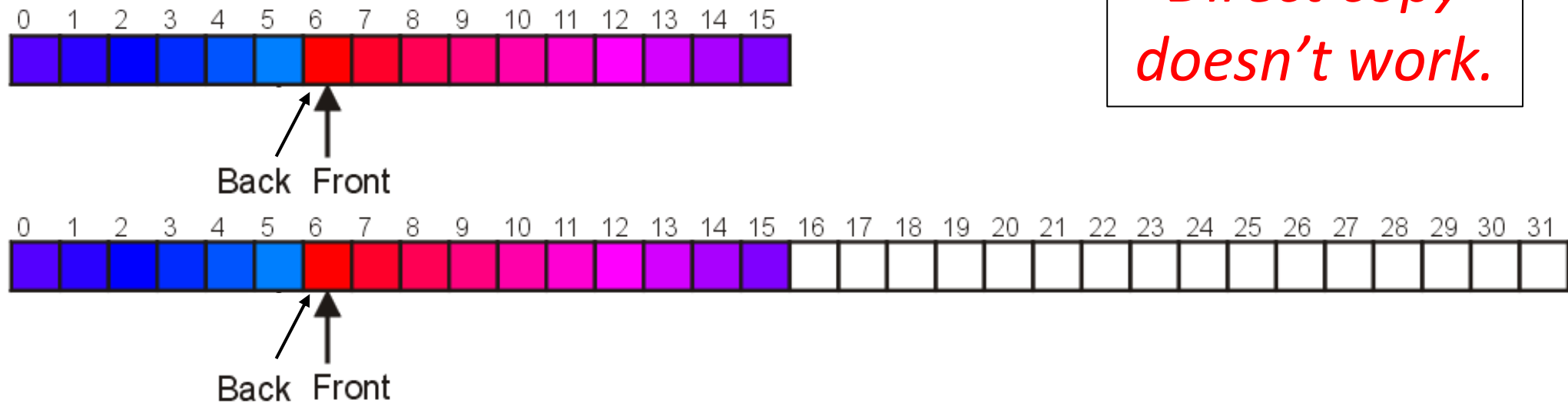
Increasing capacity

If we want to increase capacity, this becomes complex.



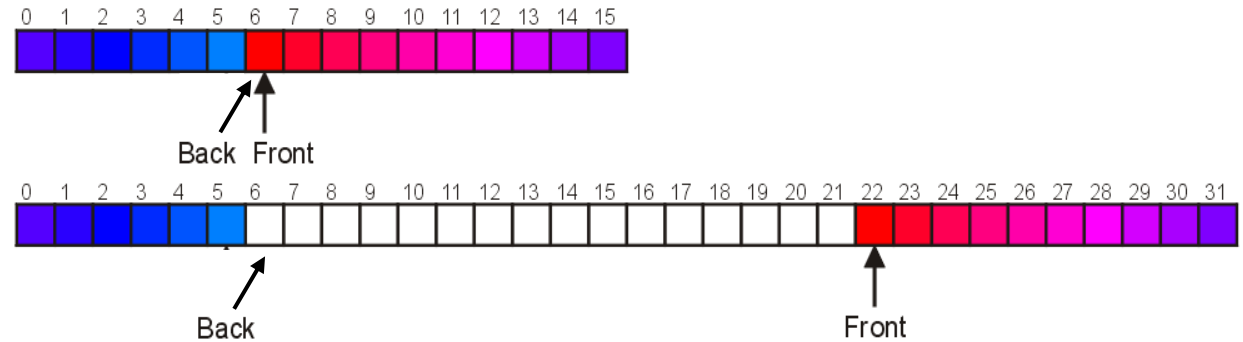
Increasing capacity

If we want to increase capacity, this becomes complex.



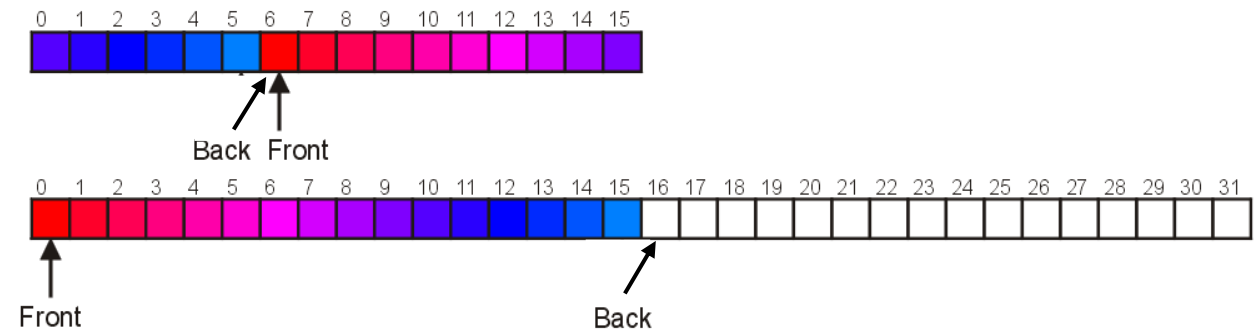
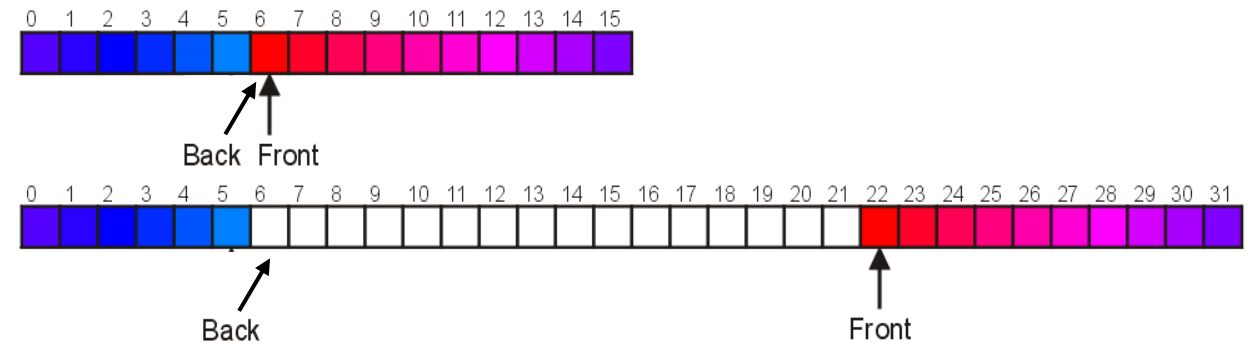
Solutions:

1. Move items beyond the front to the end of the array; the next push occurs at position#6.



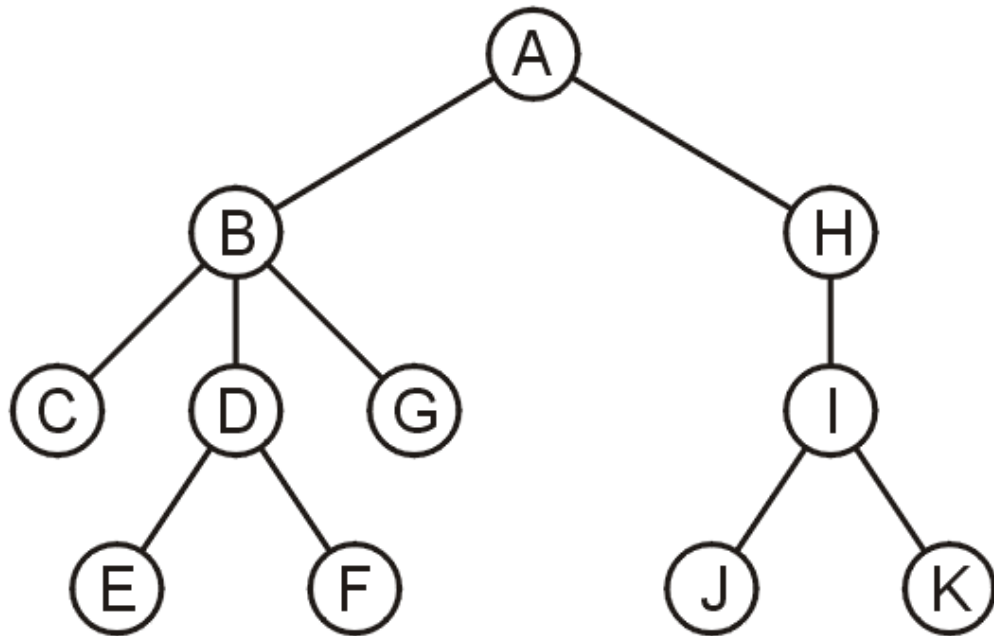
Solutions:

1. Move items beyond the front to the end of the array; the next push occurs at position#6.
2. Alternatively, map the 'front' again at position#0; the next push occurs at position#16.



Application: Breath-first Traversal

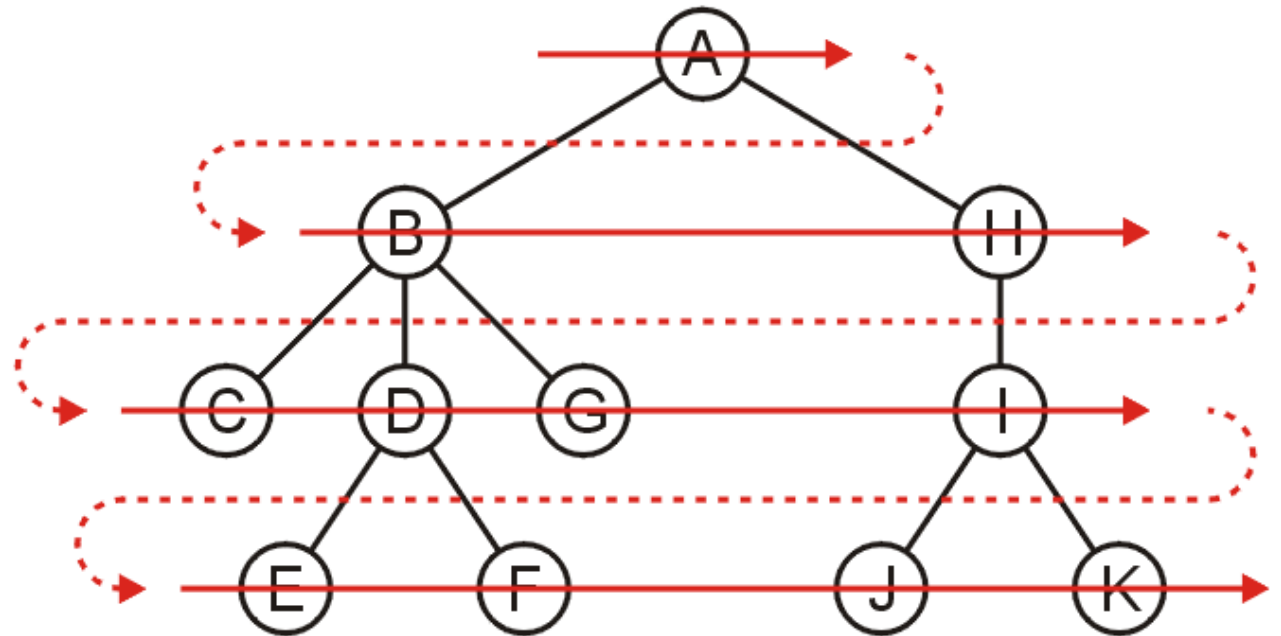
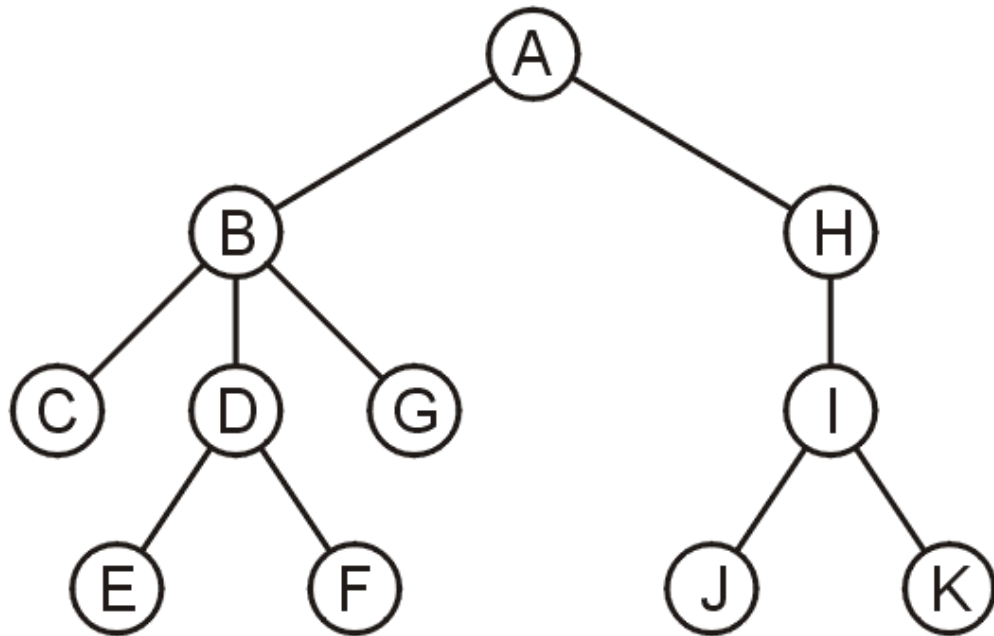
- Queue can be used for Breadth-first traversal of a directory Tree.



Application: Breath-first Traversal

- Queue can be used for Breadth-first traversal of a directory Tree.

Search all the directories at one level before descending



Breath-first Traversal

- Implementation approach:

- ???

- ???

Breath-first Traversal

- Implementation approach:
 - Place the **root** into a queue
 - ???

Breath-first Traversal

- Implementation approach:
 - Place the **root** into a queue
 - While the queue is not empty:
 - ???
 - ???

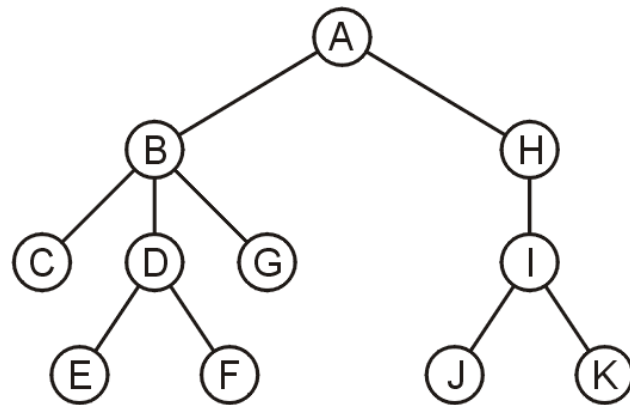
Breath-first Traversal

- Implementation approach:
 - Place the **root** into a queue
 - While the queue is not empty:
 - Pop (Dequeue) the item at the front
 - ???

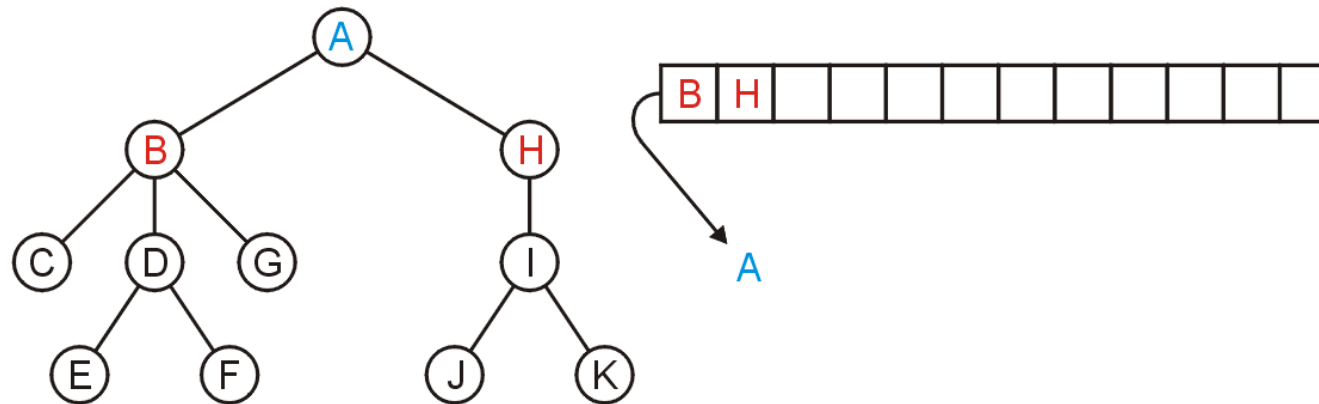
Breadth-first Traversal

- Implementation approach:
 - Place the **root** into a queue
 - While the queue is not empty:
 - Pop (Dequeue) the item at the front
 - Push (Enqueue) all its children

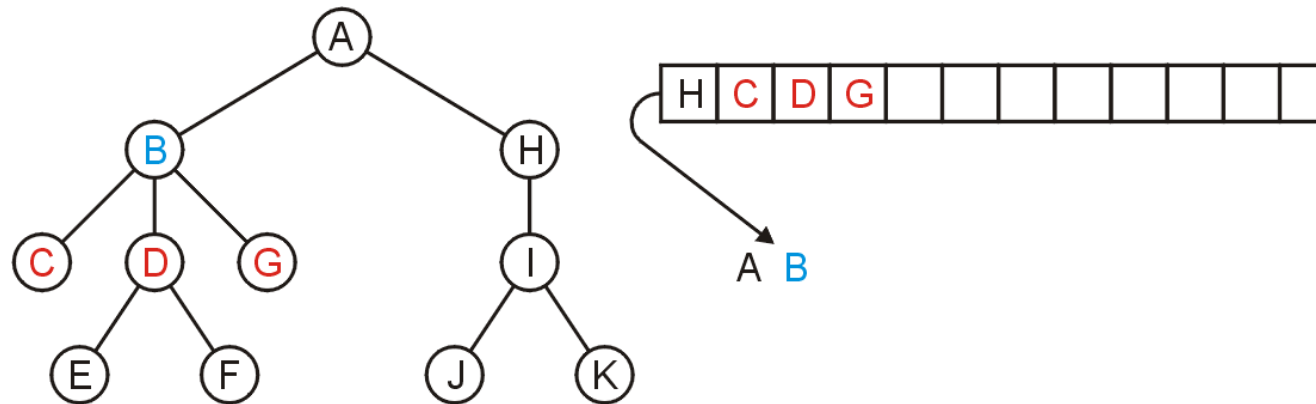
Push the root directory A



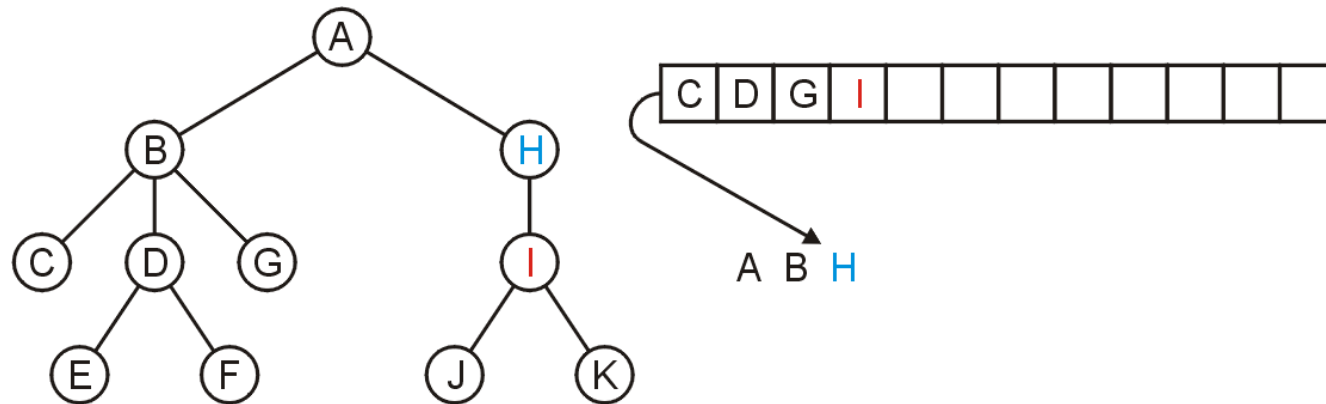
Pop A and push its two sub-directories: B and H



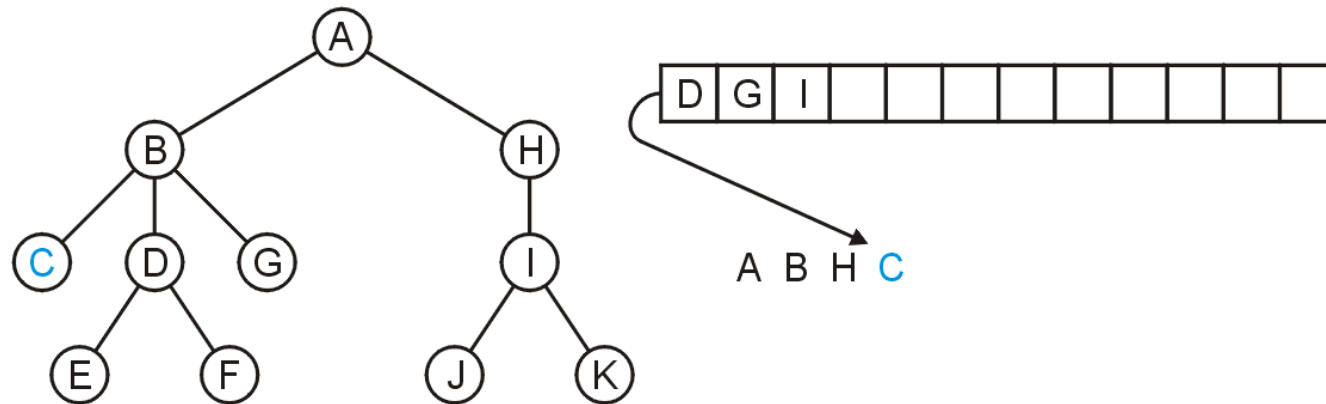
Pop B and push C, D, and G



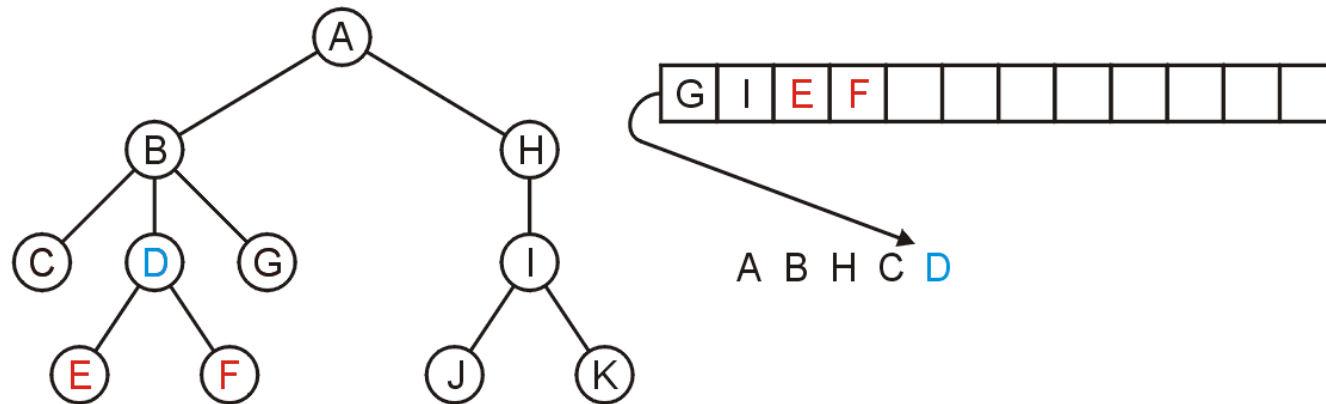
Pop H and push its one sub-directory I



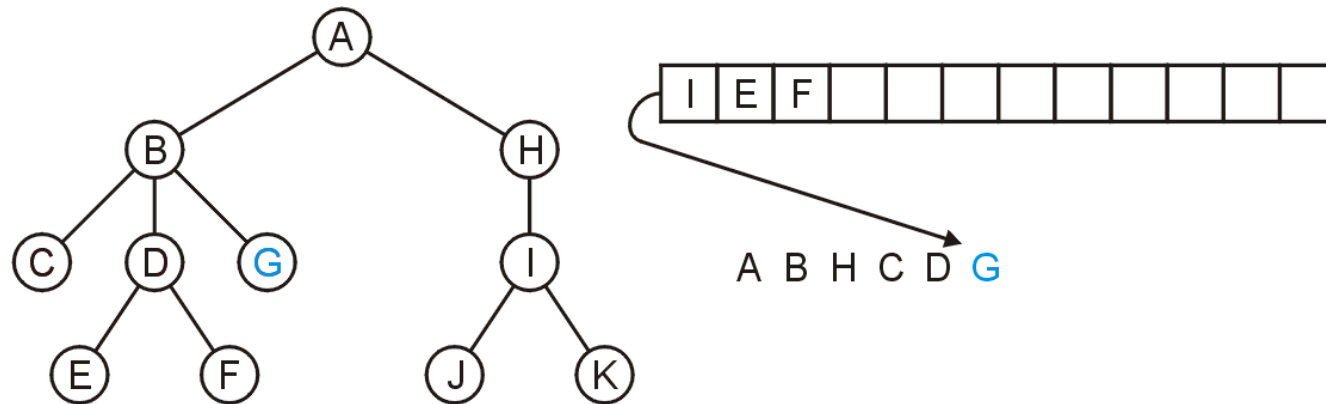
Pop C: no sub-directories



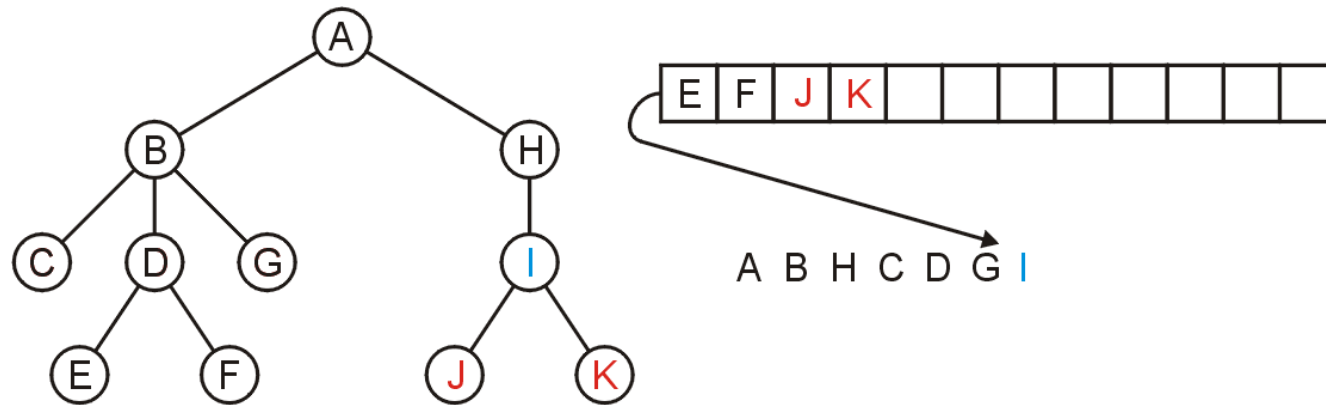
Pop D and push E and F



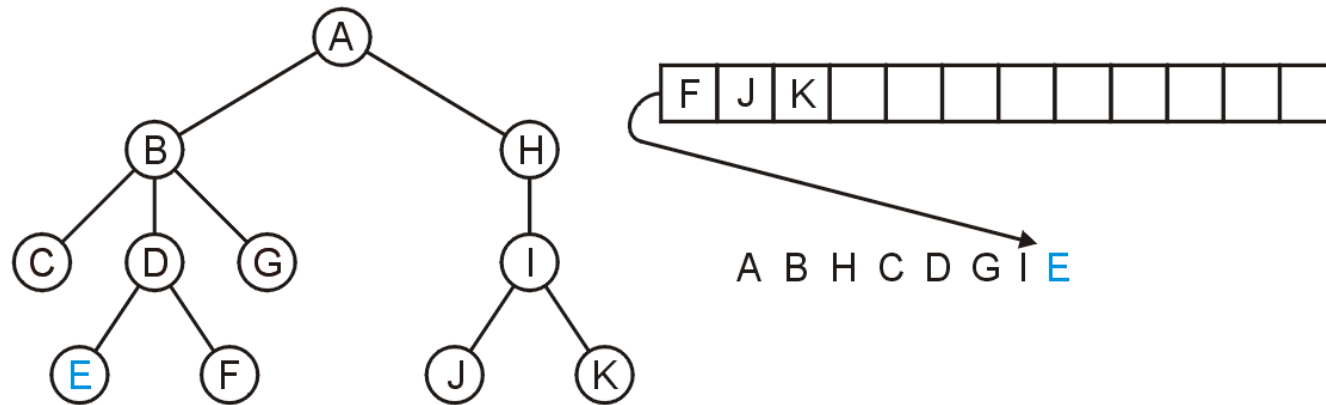
Pop G



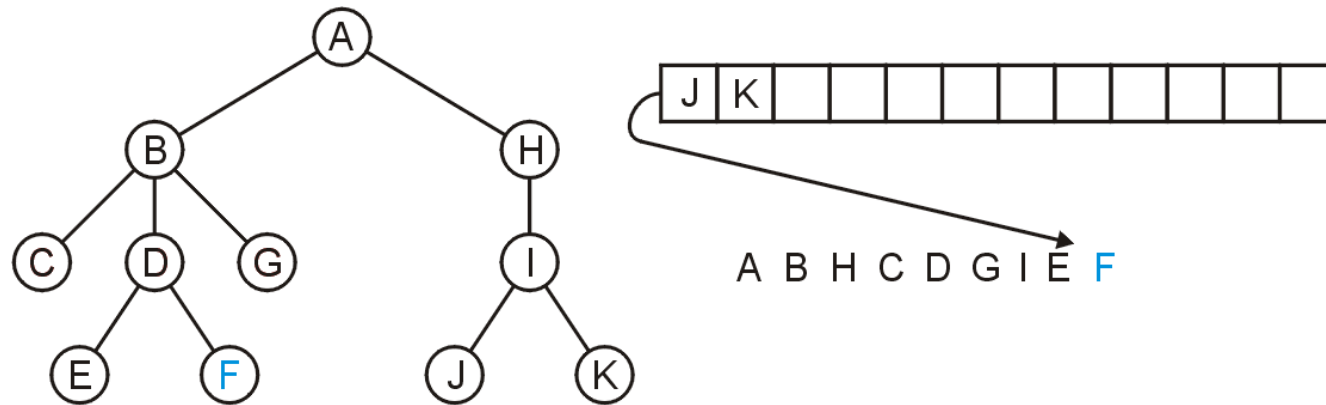
Pop I and push J and K



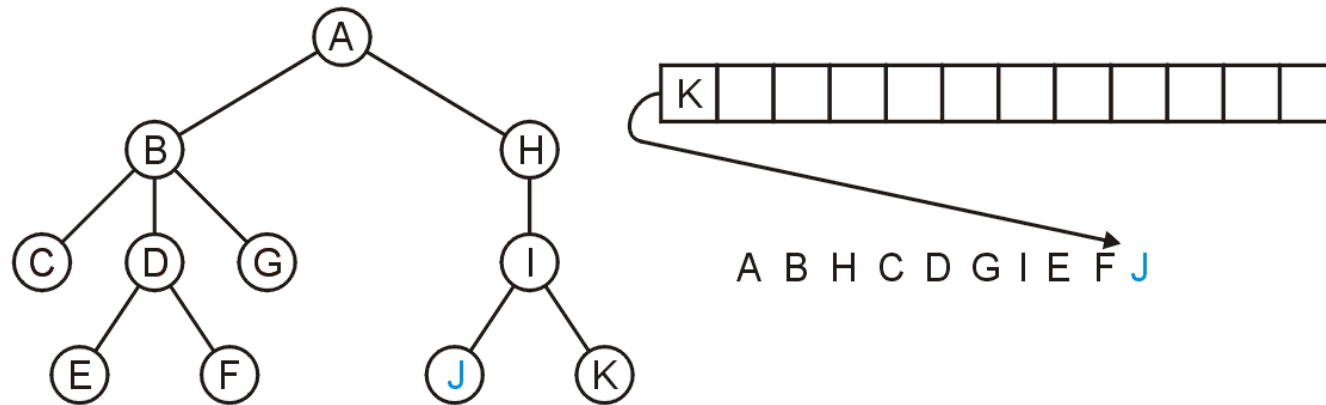
Pop E



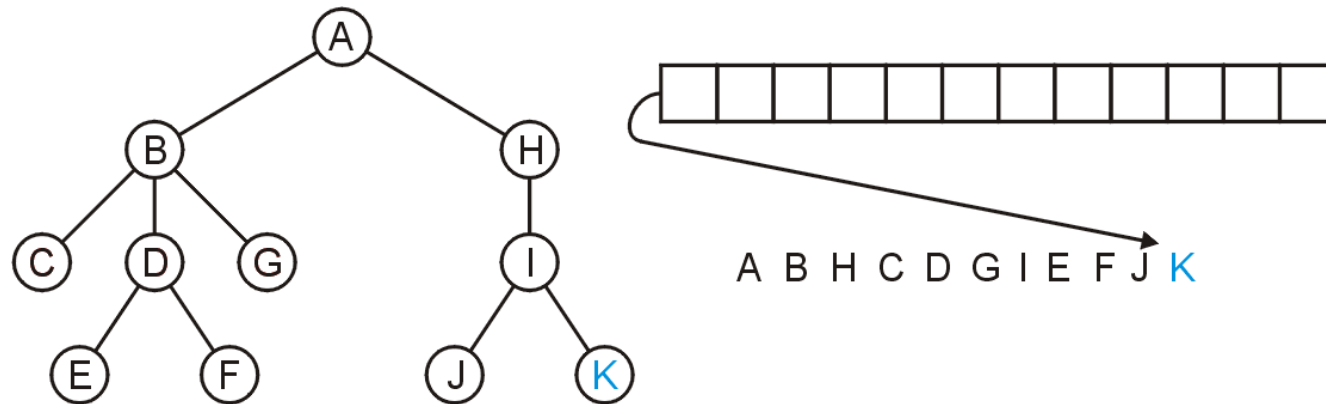
Pop F



Pop J



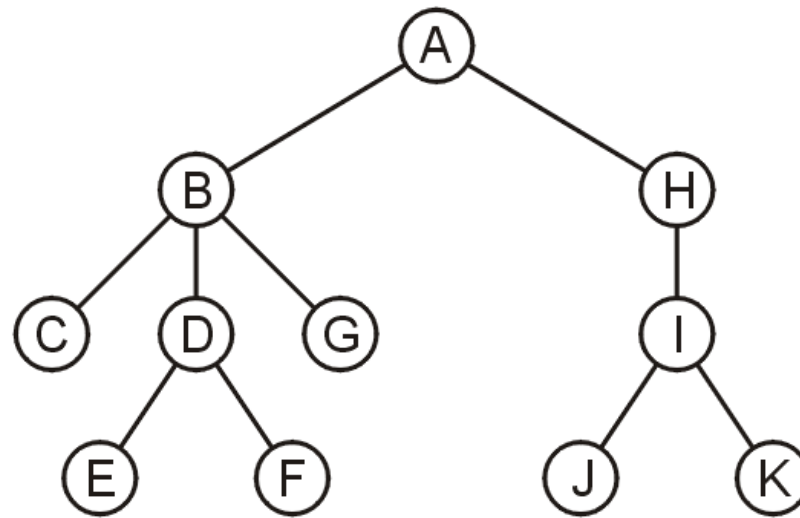
Pop K and the queue is empty



The resulting order

A B H C D G I E F J K

is in breadth-first order:



Breath-first Traversal

- Implementation approach:
 - Place the **root** directory into a queue
 - While the queue is not empty:
 - Pop (Dequeue) the directory at the front of the queue
 - Push (Enqueue) all the sub-directories into the queue

*What will be
the breadth-
first order?*

Breath-first Traversal

- Implementation approach:
 - Place the **root** directory into a queue
 - While the queue is not empty:
 - Pop (Dequeue) the directory at the front of the queue
 - Push (Enqueue) all the sub-directories into the queue

*What will be
the breadth-
first order?*

*The order in which the
directories come out
(Dequeued) of the queue*

Breath-first Traversal

*What will be
the Time
Complexity?*

- Implementation approach:
 - Place the **root** directory into a queue
 - While the queue is not empty:
 - Pop (Dequeue) the directory at the front of the queue
 - Push (Enqueue) all the sub-directories into the queue

Breath-first Traversal

- Implementation approach:
 - Place the **root** directory into a queue
 - While the queue is not empty:
 - Pop (Dequeue) the directory at the front of the queue
 - Push (Enqueue) all the sub-directories into the queue

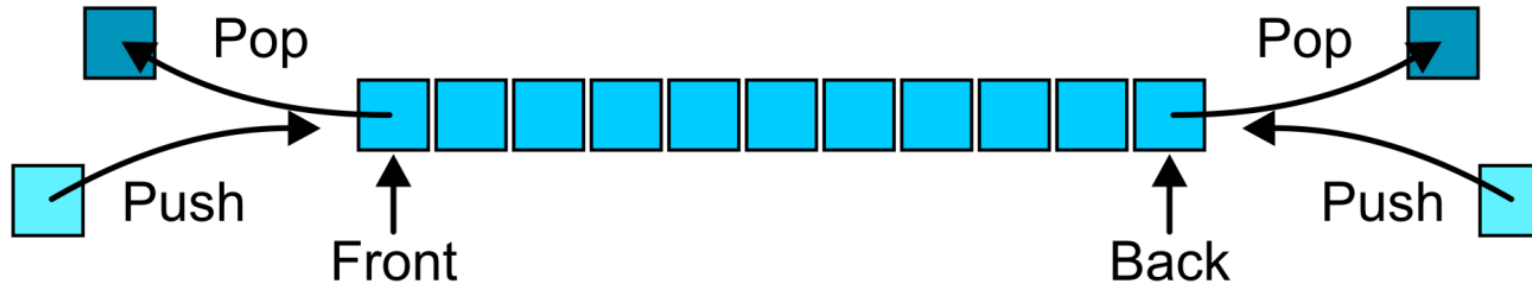
*What will be
the Time
Complexity?
 $O(\#nodes)$*

Double ended queue (Deque)

- Stores items in an explicitly defined **linear order**.
 - Items can be **pushed** onto either the **front or back**
 - Items can be **popped** from either the **front or back**

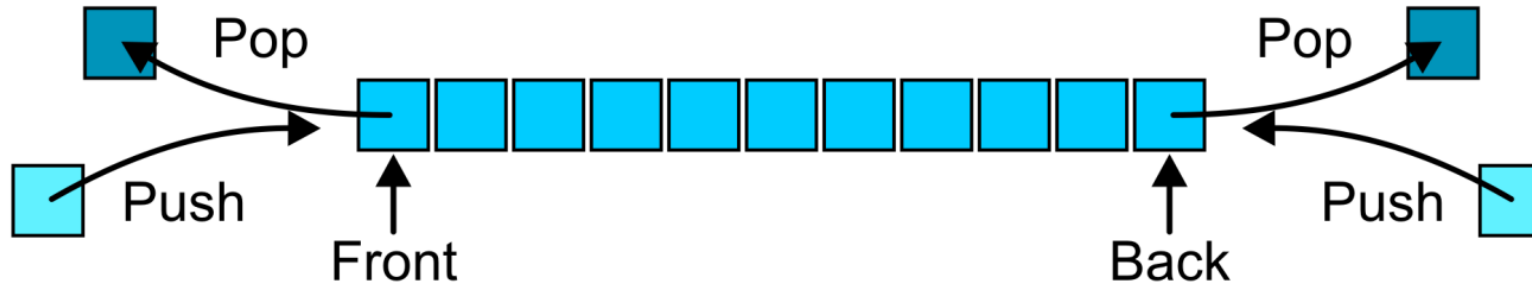
Double ended queue (Deque)

- Stores items in an explicitly defined **linear order**.
 - Items can be **pushed** onto either the **front** or **back**
 - Items can be **popped** from either the **front** or **back**



Double ended queue (Deque)

- Stores items in an explicitly defined **linear order**.
 - Items can be **pushed** onto either the **front** or **back**
 - Items can be **popped** from either the **front** or **back**



Operations:

- `Front()`, `Back()`
- `Push_front()`, `push_back()`
- `Pop_front()`, `pop_back()`

Meme alert!

LIFO





LIFO



FIFO





- Useful as a general-purpose tool:
 - Can be used as either a queue or a stack
 - Palindrome Checker

- Useful as a general-purpose tool:
 - Can be used as either a queue or a stack
 - Palindrome Checker (How?)

- Useful as a general-purpose tool:
 - Can be used as either a queue or a stack
 - Palindrome Checker (How?)
- Two major ways of implementing Deques:
 - **Requirement:** All operations must be done in $\theta(1)$
 - ???
 - ???

- Useful as a general-purpose tool:
 - Can be used as either a queue or a stack
 - Palindrome Checker (How?)
- Two major ways of implementing Deques:
 - **Requirement:** All operations must be done in $\theta(1)$
 - Doubly Linked Lists
 - ???

- Useful as a general-purpose tool:
 - Can be used as either a queue or a stack
 - Palindrome Checker (How?)
- Two major ways of implementing Deques:
 - **Requirement:** All operations must be done in $\theta(1)$
 - Doubly Linked Lists
 - ???

Can we use a singly linked list here?

- Useful as a general-purpose tool:
 - Can be used as either a queue or a stack
 - Palindrome Checker (How?)
- Two major ways of implementing Deques:
 - **Requirement:** All operations must be done in $\theta(1)$
 - Doubly Linked Lists
 - ???

*Can we use a singly linked
list here?*

*(can't ; due to $\theta(1)$
restriction)*

- Useful as a general-purpose tool:
 - Can be used as either a queue or a stack
 - Palindrome Checker (How?)
- Two major ways of implementing Deques:
 - **Requirement:** All operations must be done in $\theta(1)$
 - Doubly Linked Lists
 - Circular Array

Deque

```
#include <iostream>
#include <deque>
int main(){

    deque <int> ideque;
```

Deque

```
#include <iostream>
#include <deque>
int main(){

    deque <int> ideque;
    ideque.push_front(5)
    ideque.push_back(4)
    ideque.push_front(3)
    ideque.push_back(6)
```

*What is the current state of
Deque?*

Deque

```
#include <iostream>
#include <deque>
int main(){

    deque <int> ideque;
    ideque.push_front(5)
    ideque.push_back(4)
    ideque.push_front(3)
    ideque.push_back(6)
```

*What is the current state of
Deque?*

3 5 4 6

Deque

```
#include <iostream>
#include <deque>
int main(){
```

```
    deque <int> ideque;
    ideque.push_front(5)
    ideque.push_back(4)
    ideque.push_front(3)
    ideque.push_back(6)
```

```
// state of deque: 3 5 4 6
```

Deque

Output

Is the deque empty? 0

```
#include <iostream>
#include <deque>
int main(){

    deque <int> ideque;
    ideque.push_front(5)
    ideque.push_back(4)
    ideque.push_front(3)
    ideque.push_back(6)           // state of deque: 3 5 4 6

    cout << "Is the deque empty?" << ideque.empty() << endl
```

Deque

```
#include <iostream>
#include <deque>
int main(){
```

```
    deque <int> ideque;
    ideque.push_front(5)
    ideque.push_back(4)
    ideque.push_front(3)
    ideque.push_back(6)           // state of deque: 3 5 4 6
```

```
    cout << "Is the deque empty?"      << ideque.empty()      << endl
    cout << "Size of deque"            << ideque.size()      << endl
```

Output

```
Is the deque empty?  0
Size of deque:  4
```

Deque

```
#include <iostream>
#include <deque>
int main(){

    deque<int> ideque;
    ideque.push_front(5)
    ideque.push_back(4)
    ideque.push_front(3)
    ideque.push_back(6)           // state of deque: 3 5 4 6

    cout << "Is the deque empty?"      << ideque.empty()      << endl
    cout << "Size of deque"            << ideque.size()      << endl

    for( int i=0; i<ideque.size(); i++){
        cout<< "Back of the deque"    << ideque.back()      << endl;
        ideque.pop_back()
    }
```

Output

```
Is the deque empty?  0
Size of deque:  4
Back of the deque:  6
Back of the deque:  4
Back of the deque:  5
Back of the deque:  3
```

Deque

```
#include <iostream>
#include <deque>
int main(){

    deque<int> ideque;
    ideque.push_front(5)
    ideque.push_back(4)
    ideque.push_front(3)
    ideque.push_back(6)                // state of deque: 3 5 4 6

    cout << "Is the deque empty?"      << ideque.empty()      << endl
    cout << "Size of deque"            << ideque.size()        << endl

    for( int i=0; i<ideque.size(); i++){
        cout<< "Back of the deque"    << ideque.back()        << endl;
        ideque.pop_back()
    }

    cout<< "Is the deque empty?" << ideque.empty()      << endl;
    return 0;
}
```

Output

```
Is the deque empty?  0
Size of deque:      4
Back of the deque:   6
Back of the deque:   4
Back of the deque:   5
Back of the deque:   3
Is the deque empty?  1
```

(self-study)

- Given a deque with n entries,
 - There are two methods to access an arbitrary entry from $0 \leq k < n$
 - `ideque[k]` & `ideque.at(k)`
 - The only difference is that the second option does range checking. If $k < 0$ or $k \geq n$, an exception will be thrown called 'out_of_range'
- Another mechanism for walking through the entries of deque is to use an '**Iterator**'

Meme alert!

STACK

QUEUE



STACK

QUEUE



DEQUE



References

1. Chapter 3, 'Data Structures and Algorithm Analysis in C++', Mark Allen Weiss, Fourth Edition
2. "Basics of Queues Tutorials & Notes | Data Structures | HackerEarth," HackerEarth, Sep. 02, 2016.
<https://www.hackerearth.com/practice/data-structures/queues/basics-of-queues/tutorial/>
3. https://ece.uwaterloo.ca/~dwharder/aads/Lecture_materials/3.03.Queues.pptx (Queue)
4. https://ece.uwaterloo.ca/~dwharder/aads/Lecture_materials/3.04.Dequeues.pptx (Deque)
5. https://ece.uwaterloo.ca/~dwharder/aads/Lecture_materials/3.04.Dequeues.pdf (Deque)

Thank you